



A novel deep reinforcement learning method for solving resource-constrained multi-project scheduling problem: twin delayed deep deterministic policy gradient

Sajad Soltan^a, Maryam Ashrafi^{a,*}, Ehsan Nazerfard^b

^a Department of Industrial Engineering & Management Systems, Amirkabir University of Technology, Tehran, Iran

^b Department of Computer Engineering, Amirkabir University of Technology, Tehran, Iran

ARTICLE INFO

Keywords:

Resource-constrained multi-project scheduling problem
Deep reinforcement learning
Twin-delayed deep deterministic policy gradient
Markov decision process

ABSTRACT

This paper introduces a novel reinforcement learning framework based on Twin Delayed Deep Deterministic Policy Gradient (TD3) to effectively solve the Resource-Constrained Multi-Project Scheduling Problem (RCMPSP). In the proposed framework, the scheduling problem is formulated as a Markov Decision Process (MDP). The presented approach utilizes TD3 to autonomously learn optimal scheduling policies without extensive hyperparameter tuning. This framework is the first application of TD3 for RCMPSP, contributing to the growing body of research on Deep Reinforcement Learning (DRL) for complicated scheduling problems. The model was trained on the MPSPLIB dataset to investigate its effectiveness. The results, along with the sensitivity analysis, demonstrate that the proposed framework significantly increases computational efficiency and solution quality. In particular, TD3 outperformed the Deep Deterministic Policy Gradient (DDPG) baseline and the state-of-the-art learning-based rollout metaheuristic. The improvement was statistically validated using a Wilcoxon signed-rank test, confirming the superiority of TD3. The proposed approach achieves improvements in resource allocation, leading to a reduction in project makespan. The results provide valuable managerial insights, suggesting a scalable and adaptive strategy for multi-project scheduling in dynamic environments.

1. Introduction

The RCMPSP presents a critical optimization challenge across diverse sectors, including construction, manufacturing, software development, and supply chain management (Hu et al., 2019; Zhang et al., 2024; van der Beek et al., 2024). The problem involves scheduling multiple projects competing for a finite resource set over a shared timeline. In such environments, resource allocation and task sequencing decisions can significantly impact project completion times and the organization's overall operational efficiency (Phuntsho et al., 2024; Zhao et al., 2022). The primary objective of RCMPSP is to minimize the makespan while adhering to a multitude of constraints, including resource availability, task dependencies, and project deadlines (Asadujaman et al., 2021). Due to its combinatorial nature and numerous constraints, RCMPSP is classified as an NP-hard problem. This means that as the problem grows, finding optimal solutions becomes computationally intractable. This complexity makes RCMPSP a particularly challenging situation for both researchers and practitioners

(Zhang et al., 2024; Zhentao et al., 2023; Hartmann & Briskorn, 2022; Ding et al., 2023).

Traditionally, a variety of heuristic and exact methods have been employed to tackle RCMPSP (Cai et al., 2023). While exact methods can provide optimal solutions for smaller instances, their scalability diminishes significantly as problem complexity increases (Ratajczak-Ropel, 2018). Heuristic methods, while more scalable, often yield sub-optimal solutions and require extensive parameter tuning and domain-specific knowledge, limiting their generalizability (Van Den Eeckhout et al., 2021; Van Eynde, Vanhoucke, 2020). Moreover, heuristic methods typically require significant fine-tuning of parameters and often rely on domain-specific knowledge to perform well. As such, they lack generalizability across different problem instances, which limits their effectiveness in dynamic, real-world settings where problem characteristics can change rapidly (Jiang et al., 2023; Sung et al., 2020; Chen et al., 2015).

In recent years, the rise of machine learning (ML) and, more specifically, reinforcement learning (RL) has opened new avenues for

* Corresponding author at: Department of Industrial Engineering and Management Systems, Amirkabir University of Technology, 424 Hafez Ave., Tehran, Iran.
E-mail addresses: sajadsoltan@aut.ac.ir (S. Soltan), ashrafi.mm@aut.ac.ir (M. Ashrafi), nazerfard@aut.ac.ir (E. Nazerfard).

tackling complex scheduling and optimization problems like RCMPSP. RL has emerged as a promising alternative, especially for sequential decision-making problems that can be effectively modeled using a MDP. While RL has shown success in simpler scheduling scenarios, its application to the complex multi-project environment of RCMPSP remains relatively unexplored. Applying RL, an agent learns to make optimal decisions by interacting with an environment, receiving feedback in the form of rewards or penalties, and improving its policy based on this feedback. RL has been successfully applied to various scheduling problems, including job-shop scheduling (Lei et al., 2022; Kuhnle et al., 2022; Wang et al., 2024; Zhang & Zhu, 2024), flow-shop scheduling (Zhao et al., 2024; Kim & Lee, 2023; Wang et al., 2023; Lee & Kim, 2022), and single-project resource-constrained project scheduling (RCPPSP) (Sallam et al., 2021; Cai et al., 2024; Liu et al., 2024). However, its application to the more complex multi-project environment found in RCMPSP is still relatively unexplored. This is in part due to the high-dimensional, continuous allocation involved in scheduling decisions for multiple projects, which traditional RL algorithms like Q-learning or tabular methods struggle to handle.

The growing need for more scalable, flexible, and autonomous solutions to RCMPSP has driven interest in advanced DRL algorithms. DRL combines the decision-making capabilities of RL with the powerful function approximation abilities of deep neural networks, making it possible to handle the large, complex state and action spaces inherent in RCMPSP. Among the most promising DRL approaches for continuous allocation is the TD3 algorithm. Introduced by Fujimoto et al. (2018), TD3 improves upon earlier algorithms like DDPG by addressing the issue of overestimation bias in Q-value estimation, stabilizing policy updates, and smoothing the target policy (Chen et al., 2023). These enhancements make TD3 a robust candidate for solving RCMPSP, as it can efficiently navigate the trade-offs between task scheduling, resource allocation, and makespan minimization in continuous action environments. Despite its potential, the application of TD3 to RCMPSP has not been fully explored in the existing literature, leaving a gap for innovative research in this area.

This paper bridges a critical gap in the literature by introducing a novel framework that employs the TD3 algorithm to effectively address the RCMPSP. The framework overcomes limitations of traditional heuristic and exact methods by offering a scalable, efficient approach that autonomously learns effective scheduling policies, adapts to diverse project environments, and reduces computational effort. Validated on benchmark datasets such as MPSPLIB, the method demonstrates superior performance in both solution quality and efficiency. By pioneering the application of TD3 to RCMPSP, which excels in handling continuous allocation and improves over earlier RL algorithms, this research introduces a state-of-the-art RL technique to address the complexities of RCMPSP, contributing significantly to the literature on deep reinforcement learning in multi-project scheduling.

This research makes the following key contributions:

- 1) **Novel Application of TD3 to RCMPSP:** To the best of our knowledge, no prior research has applied the TD3 algorithm to solve the RCMPSP. This introduces a novel application of deep reinforcement learning to multi-project scheduling.
- 2) **Enhanced Performance:** Our TD3-based framework shows competitive performance compared to traditional heuristics and learning methods reported in the literature, achieving both higher-quality solutions and faster computation times. By leveraging TD3's continuous allocation and delayed policy updates, we can achieve more efficient resource allocation.
- 3) **Real-World Validation:** The proposed framework is rigorously evaluated using widely recognized benchmark datasets. MPSPLIB is utilized to assess the framework's effectiveness in project scheduling. The results demonstrate that the framework is practical and effective in real-world scheduling scenarios and is successfully applied to

MPSPLIB instances with different problem sizes, highlighting its adaptability and robustness.

The remainder of this paper is organized as follows: [Section 2](#) reviews relevant literature, encompassing traditional RCMPSP solution methods and recent advancements in applying RL to project scheduling problems. [Section 3](#) presents our proposed TD3-based framework, detailing the specific adaptations made to the algorithm for RCMPSP. [Section 4](#) discusses the experimental results, comparing the performance of our approach with existing methods. [Section 5](#) concludes the paper and suggests potential avenues for future research.

2. Literature review

In this section, the literature on the RCMPSP is reviewed based on the extended versions of the problem and solution methods. Sub-section 2.1 presents a brief overview of the latest research in the domain of characteristics of the RCMPSP models. We also provide a thorough analysis of different types of solutions applied to the RCMPSP in sub-section 2.2. Finally, the gap in the literature is discussed comprehensively in sub-section 2.3, which will be the focal point of this work.

2.1. The RCMPSP variants

The RCMPSP has a crucial role for researchers because of its applications in various fields such as construction, manufacturing, and software development, where effective scheduling of multiple interdependent projects is important for optimizing resource utilization, minimizing costs, and reducing delays (He et al., 2023). Thus, the RCMPSP a central role in project scheduling (Pellerin et al., 2020). However, due to the assumptions of RCMPSP, this problem cannot address real-world problems (Türk et al., 2025). Hence, the variants of the RCMPSP have been taken into consideration by many scholars (Li et al., 2024). The extensions of the RCMPSP model can be categorized into two primary classes, named activity-related and resource-related variants. The following subsections summarize the characteristics of the RCMPSP models developed in the literature.

2.1.1. Activity-related variants

One feature of the RCMPSP is that multiple execution modes can exist. In other words, activities have various durations and resource consumption amounts. This characteristic has been extended by Sadeghloo et al. (2024), Xing et al. (2025), Farahmand-Mehr and Mousavi (2025), and Ahmeti and Musliu (2025) to manage the availability of resources demanded by the activities in multi-project environments. Another variant related to activities in the RCMPSP model is to allow activities to be interrupted due to the activity preferences. This variant was addressed in research conducted by Joo and Chua (2017) and Fradi and Benali (2024), and can lead to considerable improvement in resource usage in time periods where resources could remain idle. Ultimately, considering various precedence relations between activities in the RCMPSP is recognized as an activity-related variant. In addition to Finish-to-Start (type), other relations, including Start-to-Start (SS), Start-to-Finish (FS), and Finish-to-Finish (FF), might exist between the activities in the practical context of multi-project scheduling. This variant, known as the generalized precedence relations (GPR), was reviewed by Schnell and Hartl (2016), Su et al. (2019), and Goudarzi et al. (2024).

2.1.2. Resource-related variants

Efficient use of resources is a growing concern in multi-project scheduling (Liu et al., 2025). As a result, there are various resource-based variants introduced by researchers. In some cases, project resources are assumed to be nonrenewable. To put it better, once some resources are assigned to an activity, they are not renewed (Zhao et al., 2024). This feature has been further studied by Zhang & Chen (2018), Araujo et al. (2020), and He et al. (2022). Another extension of the

RCMPSP is that there are some local resources in multi-project scheduling, which have been investigated recently by Fu & Zhou (2021), You et al. (2024), Kong and Chen (2025), Asadujjaman et al. (2024), and Hu et al. (2025). In this line of research, some resources are assumed to belong to a certain project, like fixed facilities. Contrasted with global resources, these resources can only be utilized by the activities of that project and not by any activity from any other project.

2.2. The RCMPSP solution approaches

RCMPSP is recognized as an NP-hard problem that has therefore attracted a large amount of research interest because of its practical relevance and computational difficulty (Hauder et al., 2020). Various solution methodologies have been developed to address this challenge, each with its strengths and limitations. The approaches can be classified into three main categories: exact methods, approximate methods, and RL (including MDP, DRL, and others) methods. The following subsections provide a comprehensive review of the literature, categorizing studies into exact, approximate, and RL methods to highlight the evolution of approaches and identify gaps that this research seeks to address.

2.2.1. Exact methods for solving RCMPSP

Exact solution algorithms are developed to guarantee optimal solutions to the RCMPSP by exhaustively exploring the solution space, usually using mathematically rigorous approaches. These algorithms are based on optimization theory and consider all schedules and resource allocations that are possible, such that the optimal solution is derived. Speranza and Vercellis (1993) developed a model-based method based on a hierarchical two-stage decomposition to solve nonprimitive multi-project management problems. Song et al. (2017) employed constraint programming (CP) to extend the result of a greedy allocation process in their solution approach towards the decentralized resource-constrained multi-project scheduling problem (DRCMPSP). The same approach was employed in research by Liu and Lu (2019). They applied the CP to lower resource dependencies as well as enhance resource allocation decisions between prefabrication projects. Krüger and Scholl (2010) included transfer times and transfer costs in the standard model of RCMPSP and used mixed-integer programming (MIP) solvers to solve the model proposed. Toffolo et al. (2016) utilized integer programming (IP) in their solution strategy to address the multi-mode resource-constrained multi-project scheduling problem (MRCMPSP). Moreover, dynamic programming (DP) has been used as an exact method in solving the multi-project scheduling problem, as illustrated in research by Zhao & Xu (2021) and Satic et al. (2024). While exact methods are effective for small problem instances, they become prohibitively expensive and difficult to solve to optimality for larger, real-world cases due to the exponential growth of the solution space (Araujo et al., 2020). Consequently, exact methods are typically combined with other approaches to address practical-scale scheduling problems.

2.2.2. Approximate methods for solving RCMPSP

Approximate algorithms are widely employed to solve RCMPSP due to the computational nature of the problem and the constraints of exact approaches. These methods do not provide optimal solutions but attempt to achieve good-quality solutions within a short period, and thus, they are suitable for large-scale, real-world instances. Over the past few years, various articles have been presented concerning the solution of RCMPSP based on approximation techniques, specifically heuristic and metaheuristic algorithms. For instance, Uysal et al. (2021) developed a new method based on a genetic algorithm (GA) for solving RCMPSP in large-scale problems. The GA is also developed by Bredael and Vanhoucke (2024), Nekouieian et al. (2025), and Chao et al. (2026). Moreover, Satic et al. (2022) modeled the stochastic RCMPSP as an MDP. They employed a priority rule-based (PR) method in the solution process. The PR method is also utilized by Hu et al. (2025). He et al.

(2021) designed simulated annealing (SA), Tabu search (TS), and another method comprising both previous methods to solve RCMPSP. In the paper presented by Chen and Ju (2015), swarm intelligence (SI) and heuristic priority rules were selected for solving the MRCMPSP. Geiger (2017) proposed a solution method based on the variable neighborhood search, accompanied by iterated local search for the MRCMPSP. Furthermore, Li and Xu (2018) introduced a new multi-agent system (MAS) for addressing the RCMPSP with distributed resources. Sun et al. (2021) applied ant colony optimization (ACO) to solve the RCMPSP. They also used a series schedule generation scheme (SSGS) to improve the ACO. Moreover, due to the high complexity level of multi-project scheduling, hybrid algorithms are the most utilized for solving the RCMPSP (Sánchez et al., 2023). For instance, Fu and Zhou (2021), Issa et al. (2021), Van Eynde and Vanhoucke (2020), Javanmard et al. (2022), Ahmeti and Musliu (2025), and Farahmand-Mehr and Mousavi (2025) used hybrid algorithms to improve solution efficiency in the context of the RCMPSP. However, approximate approaches aim to give near-optimal solutions within a reasonable time. These methods balance the accuracy of the solution and scalability and are best applicable to handle complicated instances of large-scale RCMPSPs. Additionally, approximate methods require careful parameter tuning and often rely on problem-specific knowledge to achieve acceptable performance. Moreover, their performance can diminish as the problem complexity grows.

2.2.3. Reinforcement learning methods for solving RCMPSP

The application of RL to solve scheduling problems has gained significant attention over the past several years, mostly owing to the complexities and computational challenges involved in solving these problems using traditional methods. Although RL falls in the category of approximate methods and does not guarantee globally optimal solutions (Dong et al., 2020), its flexibility and widespread application in solving project scheduling problems warrant a detailed discussion in a separate sub-section of this study. Several researchers have integrated RL within approximate algorithms. Choi et al. (2007) applied a Q-learning approach in the context of MDP to achieve optimal decision-making in the resource allocation for RCPSP with new project arrivals. Jędrzejowicz and Ratajczak-Ropel (2014) employed RL to manage resource allocation policy in RCPSP using asynchronous agents. They used the proposed algorithm to solve the multi-modal RCPSP in another research (Jędrzejowicz and Ratajczak-Ropel, 2015). Li and Womer (2015) presented approximate dynamic programming to solve RCPSP considering uncertainty in task durations. The proposed approach has been utilized for various extensions of RCPSP, like dynamic resource leveling with multiple uncertainties (Li et al., 2023), stochastic decentralized multi-project scheduling problem with resource transfers (SDRCMPSP-TT) (Zhao & Xu, 2021), and RCPSP with uncertain resource availabilities (SRCPSP-URA) (Xie et al., 2021). Shen et al. (2018) proposed a multi-objective two-archive memetic algorithm based on Q-learning (MOTA-MAQ) for solving project scheduling problems in a dynamic environment of software development. Sallam et al. (2021) applied RL to utilize the strength of both multi-operator differential evolution (MODE) and discrete cuckoo search (DCS) algorithms in stochastic RCPSP (SRCPSP). In another study conducted by Yang et al. (2024), a new algorithm for solving MRCMPSP based on Nondominated Sorting Genetic Algorithm II (NSGA II) and Q-learning was proposed. Cai et al. (2024) designed a solution method based on RL and a graph neural network (GNN) for the RCPSP with resource disruptions (RCPSP-RD). Liu et al. (2024) presented a solution method based on a multi-step double deep Q-network (MDDQN) and tabu search in a single-project scheduling environment. Chen et al. (2019) integrated RL within the Assigning Policy (RLAP) approach for multi-project scheduling. They used a dynamic state clustering (DSC) method and a feasible schedule generation schema (FSGS) in a cloud manufacturing system. Moreover, Satic et al. (2022) introduced an approach for modeling dynamic and stochastic RCMPSP using a discrete-time Markov decision process. They solved the proposed model using dynamic programming (DP).

2.3. Research gaps and contributions

The extensive research on solving the RCMPSP has demonstrated a variety of methods. The previous studies on the RCMPSP are summarized in Table 1. Exact methods can optimally apply to only small instances. Moreover, although metaheuristic methods are generally more scalable than exact approaches and can handle larger instances, they often still incur significant computational effort and require substantial instance-specific parameter tuning to obtain high-quality solutions (Hartmann & Briskorn, 2022; Sánchez et al., 2023).

RL has been recognized as a promising alternative for complex scheduling problems. Its advantages include: (i) once trained, an RL agent can generate solutions with near real-time inference and generalize across multiple instances; (ii) actor–critic architectures naturally handle continuous decision spaces, and (iii) RL can dynamically adapt to new constraints without full re-optimization. Recent studies have shown the capability of RL in a multi-project environment. However, most prior RL applications have focused on single-project scheduling, leaving continuous allocation in multi-project environments largely unexplored.

Furthermore, earlier RL implementations, such as DDPG, are prone to training instability and overestimation bias in continuous domains. These limitations have been well-documented in the actor–critic literature, where multiple studies have demonstrated that DDPG tends to overestimate Q-values and exhibits unstable convergence behavior in continuous-control settings (Fujimoto et al., 2018; Pan et al., 2020). To address these gaps, this research proposes the first application of the TD3 algorithm to the RCMPSP. The novelty lies in formulating the problem as a continuous-allocation MDP that captures dynamic resource competition across multiple projects. By leveraging TD3's twin critics, delayed policy updates, and target smoothing, the method achieves stable convergence, fine-grained resource allocation, and improved scalability compared to both metaheuristics and earlier RL methods. This directly addresses the gaps summarized in Table 1, where existing RL-based scheduling methods either rely on DDPG or discrete-action approaches and do not model continuous global resource allocation in multi-project environments.

In contrast to metaheuristics, which must be restarted for each instance, RL amortizes a one-time training cost and subsequently produces schedules in seconds. This leads to offering a decisive advantage in scenarios requiring frequent replanning. In summary, this study addresses the identified gaps by formulating a continuous-allocation MDP for multi-project scheduling with global resources, tailoring TD3 to achieve stable and scalable learning in this setting, and validating the framework on both benchmark instances and a large multi-project case study. Previously, no literature has applied TD3 to the RCMPSP.

3. Model

3.1. Problem description

Fig. 1 presents the structure of the multi-project scheduling problem considering limited resources. The RCMPSP is a challenging optimization problem that entails the scheduling of multiple interrelated projects competing for shared limited resources. Each project consists of a set of activities, where each activity has a predefined duration, a resource need, and precedence constraints that dictate the order in which activities should be completed. The main goal of RCMPSP is to minimize the overall makespan, defined as the total time required to complete all projects, while ensuring that resource usage does not exceed availability at any time. This problem is further complicated by the dynamic character of resource allocation, the need to manage competing priorities among projects, and the potential for inter-project dependencies. The proposed approach is based on the following assumptions:

- Activities adhere to a finish-to-start sequence.
- All resources are global and renewable.

- A singular mode of execution is available for activities.
- There is no allowance for substituting one resource for another.
- All parameters are assumed to be deterministic.
- The overarching managerial aim is to minimize the makespan of all projects.

To complement the classical definition, this study introduces an extended formulation of the RCMPSP that emphasizes its dynamic, multi-project nature. In particular, the problem is expressed in a step-wise decision-making structure that tracks activity progress, resource interactions across projects, and real-time feasibility conditions. This enriched representation provides the foundation for the MDP formulation introduced in the following subsection, enabling the problem to be treated as a sequential allocation process rather than a static optimization model (Fig. 2). This structural adaptation is essential for supporting continuous resource-allocation decisions and scalable learning within the proposed TD3-based framework.

3.2. MDP formulation

RCMPSP entails a sequential decision-making process to assign limited resources to multiple projects, considering precedence constraints and minimizing makespan. Every decision impacts the availability of resources and project schedules, so the process is dynamic and conditional on the current state. The RCMPSP also requires decisions in continuous allocation, like determining the precise allocation of resources to activities or adjusting schedules dynamically. Hence, allocating resources and the subsequent scheduling of multiple projects can be modeled as an MDP with continuous allocation. The definition of MDP's components for the proposed method is shown in Table 2.

3.3. Deep reinforcement learning

RL is a subset of ML algorithms where an agent learns to take actions by interacting with its environment to maximize rewards. DRL is a branch of ML that combines RL principles with deep neural networks to enable agents to learn optimal decision-making policies from high-dimensional inputs. By interacting with the environment and using deep learning models to approximate value functions or policies, DRL enables agents to independently discover complex strategies (Mnih et al., 2015). The main advantages of DRL methods in multi-project scheduling are threefold: (i) they are data-driven and able to determine optimal resource allocation policies in real-time, even within complex scheduling environments (Gao et al., 2022), (ii) due to the model-free nature, the agent learns optimal resource allocation policies by interacting with the environment without any explicit knowledge and a rigorous mathematical model (Kim and Lee, 2020), and (iii) they possess self-adaptability and collaborate online without needing prior forecast information about the scheduling environment (Cao et al., 2020).

3.4. Deep deterministic policy gradient

DRL has led to the development of various value-based algorithms, such as Deep Q-Networks (DQN) (Rajasekhar et al., 2024), as well as policy-based algorithms, including DDPG (Samende et al., 2022). The DDPG algorithm is a combination of the Deterministic Policy Gradient (DPG) algorithm and deep neural networks (Dong et al., 2020). Additionally, DDPG serves as an extension of the DQN algorithm for handling continuous allocation, a challenge that DQN cannot directly address (Sumiea et al., 2024). DDPG is the first DRL algorithm to utilize the actor-critic framework, employing neural networks to model Deterministic Policy Gradient (DPG) functions and effectively handle high-dimensional actions and states in continuous control tasks (Huang et al., 2024). DDPG can be applied in resource allocation (Zheng et al., 2022), autonomous driving (Barnawi et al., 2022), robotics (Jiang et al.,

Table 1
Literature overview on solution methods of RCMPSP.

Authors	Problem Scope		Activity-related Features					Resource-related Features				Solution Approaches			
	Single Project	Multi Project	Single Mode	Multiple Mode	Preemption allowed	Classical FS Precedence	Generalized precedence relations	Renewable Resource	Nonrenewable Resource	Global Resource	Local Resource	Exact Methods	Approximate Methods	Reinforcement Learning	Deep Reinforcement Learning
Speranza and Vercellis (1993)		✓	✓			✓		✓		✓		✓			
Choi et al. (2007)	✓		✓			✓		✓		✓				✓	
Krüger and scholl (2010)		✓	✓			✓		✓			✓	✓			
Jędrzejowicz and Ratajczak-Ropel (2014)	✓		✓			✓		✓		✓				✓	
Li and Womer (2015)	✓		✓			✓		✓			✓			✓	
Toffolo et al. (2016)		✓	✓			✓		✓		✓		✓			
Song et al. (2017)		✓		✓		✓		✓		✓		✓✓			
Shen et al. (2018)	✓		✓			✓		✓		✓				✓	
Liu and Lu (2019)		✓		✓		✓		✓		✓		✓			
Li et al. (2020)	✓		✓			✓		✓		✓				✓	
Xie et al. (2021)	✓			✓		✓		✓		✓				✓	
Zhao & Xu (2021)		✓	✓			✓			✓	✓				✓	
Sallam et al. (2021)	✓		✓			✓		✓			✓			✓	
Issa et al. (2021)		✓			✓			✓		✓			✓		
He et al. (2022)		✓	✓			✓			✓	✓			✓		
Yang et al. (2024)		✓		✓		✓		✓		✓				✓	
Fradi and Benali (2024)		✓	✓		✓			✓		✓			✓		
Satic (2024)		✓	✓			✓		✓		✓		✓			
Goudarzi et al. (2024)		✓		✓			✓	✓		✓			✓		
Breadel and Vanhoucke (2024)		✓	✓			✓		✓			✓		✓		
Sadeghloo et al. (2024)		✓		✓		✓			✓				✓		
Cai et al. (2024)	✓					✓		✓							✓
Asadujjaman et al. (2024)		✓	✓			✓		✓			✓		✓		
Ahmeti and Musliu (2025)		✓		✓		✓		✓			✓		✓		
Nekoueian et al. (2025)		✓	✓			✓		✓		✓			✓		
Liu et al. (2025)		✓	✓			✓					✓		✓		
Farahmand-Mehr and Mousavi (2025)		✓	✓			✓			✓	✓			✓		
Chao et al. (2026)		✓	✓			✓		✓		✓				✓	
This Study		✓	✓			✓		✓		✓					✓

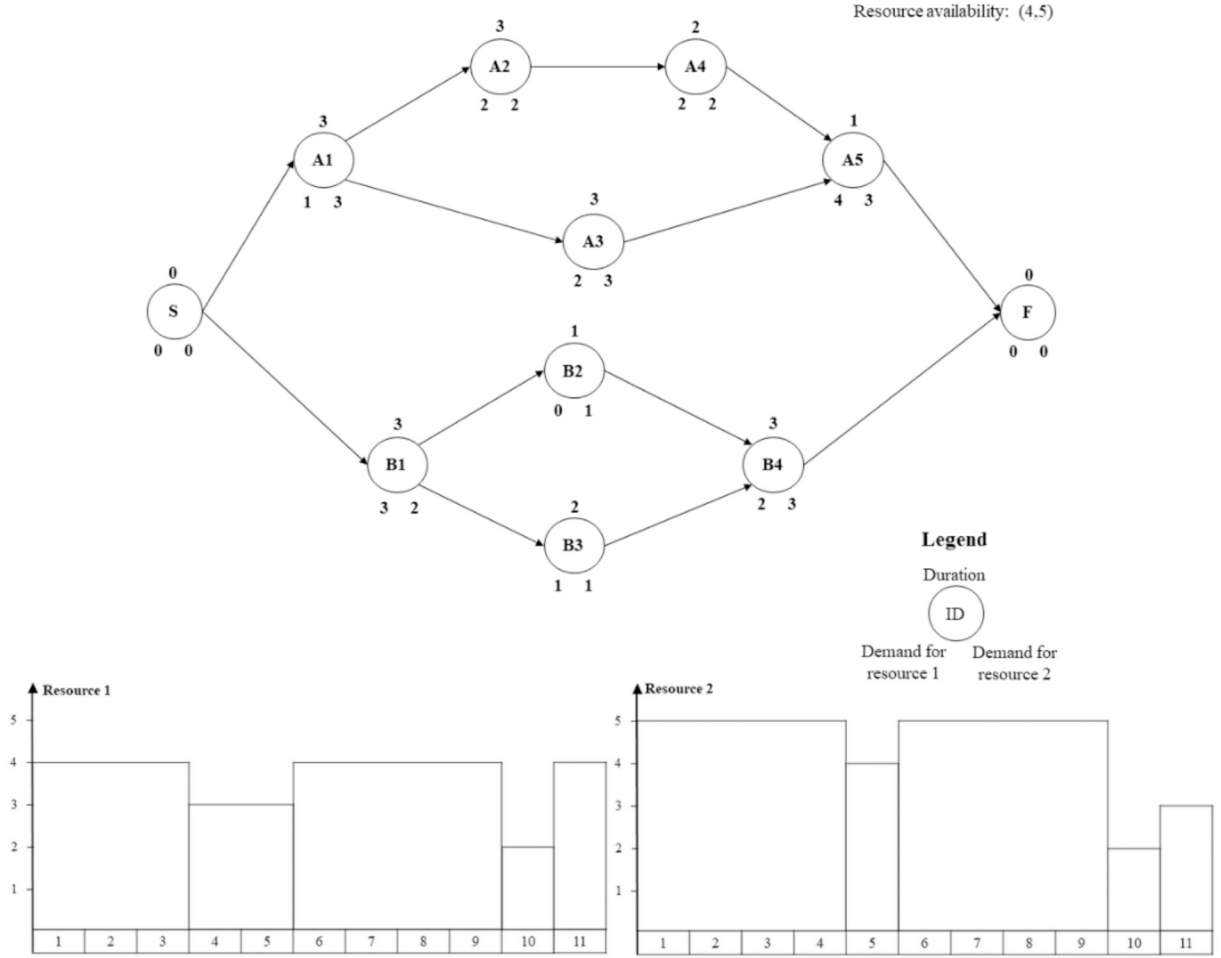


Fig. 1. Schematic view of the multi-project scheduling.

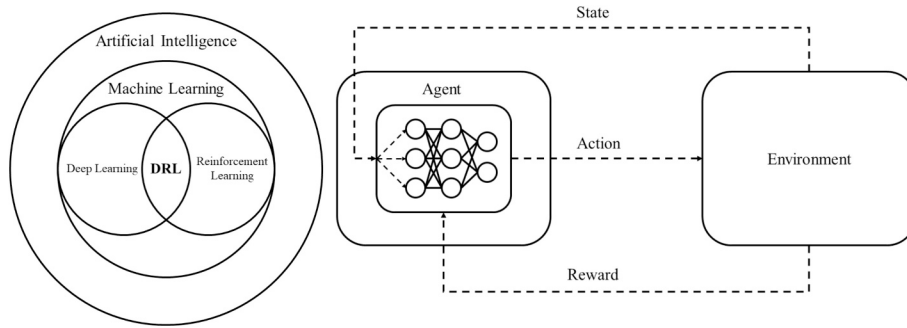


Fig. 2. A general structure of DRL.

2022), and Internet of Things (IoT) (Chen et al., 2023). DDPG integrates a policy network with a deep Q-network, requiring simultaneous learning of both networks. The policy network (actor) produces deterministic actions, while the Q-network (critic) assesses these actions to refine the policy and optimize action selection for maximizing rewards (Tan et al., 2023). The optimal policy at state s selects the action that maximizes the expected return starting from s . Consequently, if Q^* is known, the optimal action $a^*(s)$ can be determined directly using (Sumiea et al., 2024):

$$a^*(s) = \underset{a}{\operatorname{argmax}} Q^*(s, a) \quad (1)$$

where s represents the current state of the agent, a denotes the action

generated by the policy network, and $Q^*(s, a)$ refers to the hypothetical optimal value function. Moreover, Bellman's equation is utilized to iteratively estimate the optimal action-value function.

$$Q^*(s, a) = E_{s' \sim p} [r(s, a) + \gamma \max_{a'} Q^*(s', a')] \quad (2)$$

Where $s' \sim P$ indicates the next state, and s' is sampled by the environment. Suppose the approximator is a neural network $Q_\phi(s, a)$, with parameters ϕ , and we have collected a dataset D of transitions (s, a, r, s', d) , where d indicates whether the state s' is terminal. Therefore, the mean-squared Bellman error (MSBE) function can be defined to quantify how well Q_ϕ approximates the Bellman equation as follows (Dong et al., 2020):

Table 2
Definition of MDP's components.

Component	Description	Definition for the proposed method
Agent	The decision-making entity to make decisions and perform actions	It is project portfolio manager who responsible for learning and making scheduling decisions, such as selecting tasks and allocating resources dynamically
State	The set of information about environment that the agent has at a given time	The state consists of the current progress of all tasks (e.g., not started, not completed, and completed), resource availability, task durations, remaining resource demands, precedence constraints, and time elapsed in the scheduling process.
Action	The decision-making mechanism by which the agent transitions between states	It represents resource allocation and task scheduling decisions. The agent decides which task(s) to be allocated resources and how much resource to be allocated to tasks
Transition	The influence of action taken by the agent on the environment through moving from one state to the next	It is defined by the dynamic scheduling environment, where the state evolves based on task progress, resource availability updates, and the precedence constraints following the agent's action.
Reward	A numerical feedback from environment to the agent to evaluate the quality of the agent's actions in a given state	It is designed to minimize the makespan by providing a negative reward proportional to the total project completion time, encouraging the agent to complete all tasks as quickly as possible while adhering to resource and precedence constraints.

$$MSBE = E_{(s,a,r,s',d) \sim D} [(Q_\phi(s,a) - y)^2] \quad (3)$$

where the target value y is given by:

$$y = r + \gamma(1 - d)Q_{\phi'}(s', a') \quad (4)$$

where γ is the discount factor, $Q_{\phi'}$ represents the target network, a delayed copy of Q_ϕ , and a' is the action selected according to the policy for state s' .

There are two techniques employed by DDPG. (i) training deep neural networks to approximate $Q^*(s, a)$ relies on an experience replay buffer (D) that stores past experiences. The replay buffer should be large enough to capture diverse experiences but balanced to avoid overfitting to recent data or slowing learning with excessive data. Besides, the MSBE loss is minimized to make the Q-function resemble the target value, but instability arises because the target depends on the same parameters being trained (ϕ). To address this, (ii) a target network with parameters ϕ_{targ} is applied, which lags behind the primary network to provide a more stable target. The target network parameters are updated with a time delay to approximate ϕ (Sewak & Sewak, 2019):

$$\text{target} = r + \gamma(1 - d)\max_a Q_\phi(s', a') \quad (5)$$

The target network is updated after each main network update using Polyak averaging:

$$\phi_{\text{targ}} \leftarrow \rho \phi_{\text{targ}} + (1 - \rho) \phi \quad (6)$$

where ρ is a hyperparameter between 0 and 1.

3.5. Twin-delayed deep deterministic policy gradient

DDPG suffers from overestimation of value estimates caused by functional approximation errors, leading to policy instability and potential convergence to local optima (Wang, 2024). To address this issue, an improved algorithm called TD3 has been proposed by Fujimoto et al. (2018). It has been applied in various areas like fuel cell vehicle energy management (Zhang et al., 2022), hybrid electric vehicles (HEVs) (Yao et al., 2021), manipulation of multi-DOF manipulators in 3D space (Min et al., 2019), 4-ant-legged robot (Dankwa and Zheng, 2019), stock market scenario (Vishal et al., 2021), algorithmic trading (Kong & So, 2023), and automated forex trading (Chau et al., 2022). The three main tricks used in TD3 to address issues related to DDPG are (Domínguez-Barbero et al., 2023):

4. 1. TD3 adds clipped noise $\epsilon \sim N(0, \sigma)$ to the target action $a'(s')$, reducing the policy's ability to exploit Q-function errors by smoothing the Q-value across variations in action and avoiding overfitting

$$a'(s') = \text{clip}(\mu_{\phi_{\text{targ}}}(s') + \text{clip}(\epsilon, -c, c), a_{\text{Low}}, a_{\text{High}}) \quad (7)$$

Where, a_{Low} and a_{High} are the interval of the valid actions, and c is the noise range.

5. 2. TD3 addresses overestimation by learning two Q-functions and using the smaller of the two Q-values to compute targets in the Bellman error loss functions

$$y(r, s', d) = r + \gamma(1 - d)\min_{i=1,2} Q_{\phi_i}(s', a'(s')) \quad (8)$$

$$L(\phi_i, D) = E_{(s,a,r,s',d) \sim D} [(Q_{\phi_i}(s, a) - y(r, s', d))^2] \quad , \quad i = 1, 2 \quad (9)$$

6. 3. In TD3, the policy and target networks are updated less frequently than the Q-function (one policy update for every two Q-function updates is recommended). Finally, the policy is learned just by maximizing Q_{ϕ_1}

$$\max_{\theta} E_{s \sim D} [Q_{\phi_1}(s, \mu_\theta(s))] \quad (10)$$

Due to the RCMPSP's nature, TD3 can be applied to solve the multi-project scheduling problem. Fig. 3 represents the TD3 architecture for solving the RCMPSP.

6.1. Adaptation of TD3 for RCMPSP

To apply TD3 to the RCMPSP, the problem is formulated as an MDP, enabling the RL agent to autonomously learn scheduling policies. The adaptation requires defining the state space, action space, reward function, solution representation, and training procedure so that TD3 can effectively address the complexities of multi-project scheduling.

6.1.1. State-action-reward definition

The state space represents the current status of the scheduling environment. It encodes the progress of each activity (not started, in progress, or completed), precedence feasibility (binary flags indicating whether predecessors are completed), resource availability (normalized continuous values for the remaining capacity of each renewable resource), activity durations and resource requirements, and the elapsed scheduling period. This structured representation ensures that the environment can be expressed as a numeric vector suitable for neural network input. The action space defines how the agent interacts with the environment, where actions correspond to selecting feasible activities

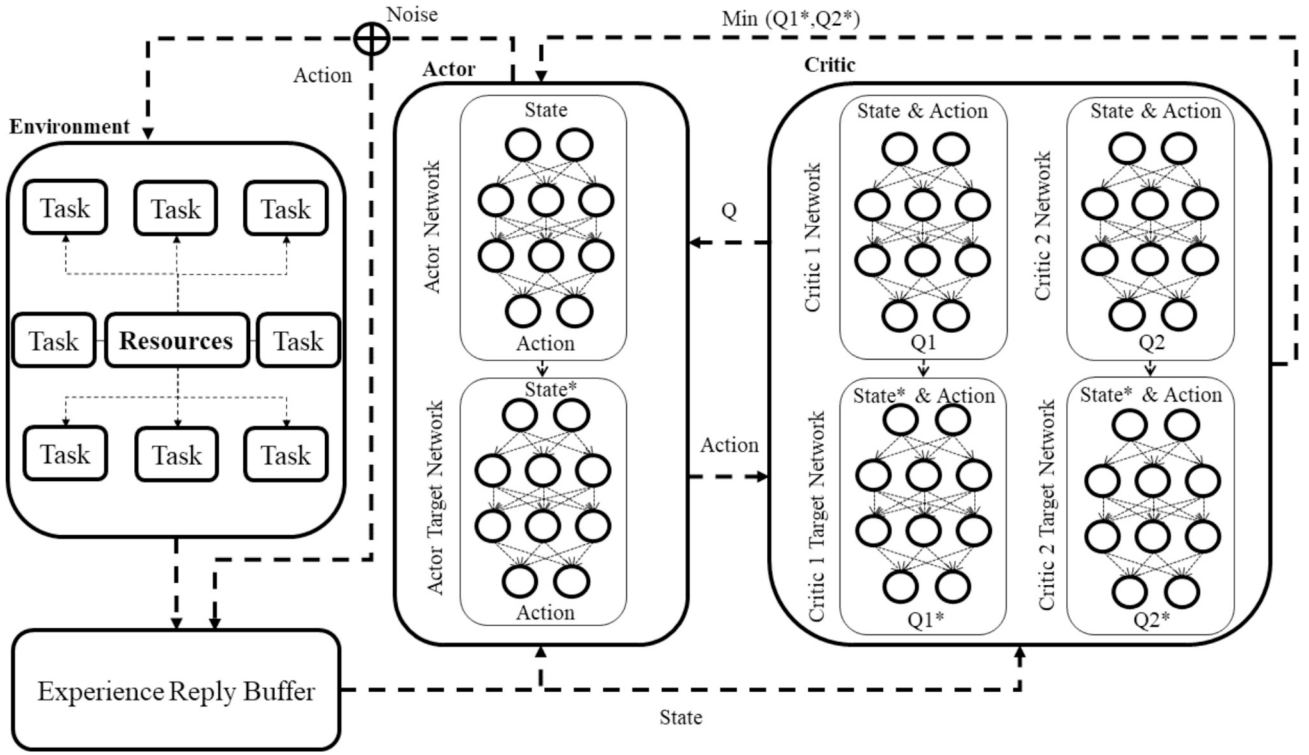


Fig. 3. Workflow of the proposed TD3-based approach.

and determining their resource allocations. The reward function is designed to guide learning by penalizing longer makespans and encouraging efficient use of limited resources. This structure ensures that the agent learns to balance precedence constraints with resource allocation trade-offs, ultimately driving improvements in scheduling efficiency.

6.1.2. Solution representation and neighborhood exploration

In the proposed framework, a scheduling solution is represented implicitly as a trajectory of state-action pairs, rather than as an explicit combinatorial structure. Specifically, a complete schedule is constructed as a sequential trajectory $\{(s_t, a_t)\}_{t=1}^T$, where s_t denotes the system state at decision step t , and the action $a_t = \mu_\theta(s_t)$ is generated by the actor policy μ_θ . The planning horizon T corresponds to the completion of all activities across all projects. The state s_t encodes the current scheduling context, including activity execution status, precedence feasibility, and remaining capacities of global renewable resources. The action a_t is a continuous resource-allocation vector that determines how available resources are distributed among feasible activities at step t . By executing the policy iteratively, the schedule emerges dynamically as the accumulation of these allocation decisions, rather than being predefined as an activity list or priority rule.

This representation fundamentally differs from classical metaheuristic formulations, where solutions are explicitly encoded (e.g., permutations, activity lists, or priority vectors) and improved through handcrafted neighborhood operators such as swap, insertion, or shift. In contrast, the proposed approach performs optimization directly in the policy space, where improvements arise from modifying the mapping $\mu_\theta : s_t \rightarrow a_t$ rather than manipulating a discrete solution structure. Although no explicit neighborhood operators are defined, neighborhood exploration is implicitly embedded in the TD3 learning dynamics. During training, stochastic exploration noise is added to the actor output, resulting in executed actions of the form $a_t = \mu_\theta(s_t) + \epsilon_t$ where ϵ_t is clipped noise. These perturbations generate local deviations around the deterministic policy output, corresponding to continuous neighborhood

moves in the resource-allocation space. Small changes in a_t induce alternative scheduling trajectories that differ locally from the current solution, analogous to local moves in metaheuristic search.

Furthermore, TD3 introduces clipped noise into the target policy used for critic updates. This mechanism smooths the value function landscape and prevents overexploitation of narrow optima, effectively encouraging exploration within a bounded neighborhood around the greedy policy. In addition, the experience replay buffer stores transitions generated under diverse policies and exploration phases. Mini-batch sampling from this buffer exposes the learning process to multiple trajectories simultaneously, enabling the algorithm to explore multiple neighborhoods in parallel, rather than following a single search path. Policy improvement is achieved through deterministic policy gradient updates, which adjust the actor parameters θ to maximize the expected return. This process shifts the entire action trajectory $\{a_t\}_{t=1}^T$ toward regions of the solution space associated with lower makespan. Conceptually, this acts as an adaptive improvement operator that continuously reshapes the neighborhood structure based on accumulated learning, rather than relying on fixed, manually designed operators. The key elements of solution representation and neighborhood generation are summarized as follows:

- **Solution representation:** A scheduling solution is represented as an ordered sequence of decisions, where each decision assigns resource fractions to feasible activities given the current system state. This trajectory-based representation replaces static encodings with a dynamic construction process, allowing the policy to adapt allocations in response to evolving project and resource conditions.
- **Neighborhood definition:** Neighboring solutions correspond to alternative action trajectories induced by small perturbations in continuous resource-allocation decisions.
- **Neighborhood generation:** Exploration noise and target policy smoothing generate bounded local variations around the current policy output.

- Search mechanism: Replay-buffer sampling enables simultaneous exploration of multiple neighborhoods across different scheduling trajectories.
- Improvement operator: Deterministic policy gradient updates iteratively shift the policy toward higher-quality schedules, serving as an adaptive neighborhood improvement process.

6.1.3. Training procedure

The training process follows the standard TD3 paradigm. The agent interacts with a simulated RCMPSP environment, generating transitions of the form (s_t, a_t, r_t, s_{t+1}) . These experience tuples are stored in a replay buffer, from which mini-batches are sampled to update the networks. This mechanism decorrelates training data and stabilizes convergence. Importantly, the training data are generated dynamically by the agent's own interaction with the environment rather than from any external dataset. As a result, the quality of learning depends primarily on the reward design and exploration strategy, and the trained policy can generalize to new problem instances without retraining from scratch.

7. Experimental results

7.1. Dataset

To assess the effectiveness, computational efficiency, and scalability of the presented TD3-based approach for solving the RCMPSP, we utilize the MPSPLIB dataset, a well-known benchmark proposed by [Hombberger \(2007\)](#). The MPSPLIB dataset, which can be found through <https://www.mpsplib.com/download>, has been used extensively in the literature to evaluate the effectiveness of various approaches in solving different types of the RCMPSP. The dataset contains 140 instances, categorized into various problem sizes to address different levels of complexity. To be specific, the single-mode RCPSP set includes four instance sets: j30, j60, j90, and j120, which represent projects with 30, 60, 90, and 120 activities, respectively. This structured classification allows for a comprehensive assessment of scheduling algorithms across different problem sizes. Given its established role as a benchmark in the field, the MPSPLIB dataset guarantees a standardized and fair comparison with current methodologies. Several scholars have relied on this dataset to validate their algorithms, including [Wauters et al. \(2015\)](#), [Villafañez et al. \(2019\)](#), [Van Eynde and Vanhoucke \(2022\)](#), [Bredael and Vanhoucke \(2023\)](#), [Cai et al. \(2024\)](#), and [Torba et al. \(2024\)](#). In this study, we focus on a specific instance from MPSPLIB to test the proposed model. The selected instance consists of 7 projects, each containing 92 activities, including two dummy activities per project, representing the start and end points. The projects compete for four global renewable resources.

7.2. Hyperparameters

The hyperparameters employed in this study were primarily adopted from the defaults recommended in [Fujimoto et al. \(2018\)](#) and widely used in open-source implementations, as these settings have been shown to provide stable performance in continuous-control environments. To determine suitable hyperparameters for the TD3 algorithm in the RCMPSP environment, a systematic tuning procedure was carried out using Optuna, an automated hyperparameter optimization library based on Bayesian optimization, introduced by [Akiba et al. \(2019\)](#) and used in many studies ([Specht and Madlener, 2023](#); [Hammler et al., 2023](#)).

7.2.1. Tuning objective and evaluation criterion

The tuning problem was formulated as a minimization of the average makespan obtained by the final trained TD3 policy over a fixed validation set of RCMPSP instances drawn from MPSPLIB. This choice aligns the tuning process directly with the primary scheduling objective and avoids optimistic bias associated with best-episode or transient peak performance. Importantly, no best-of-k episode selection was applied;

instead, only the final converged policy of each training run was evaluated, reflecting realistic deployment conditions in which a single learned policy is used repeatedly.

7.2.2. Trial structure and computational procedure

The tuning process consisted of 50 Optuna trials, where each trial corresponds to one independent TD3 training run initialized from scratch. For every trial, the TD3 agent was trained for 1,400 episodes, which was empirically sufficient to reach stable convergence. During each trial, Optuna sampled a unique combination of four key hyperparameters. The key hyperparameters to be adjusted are actor network learning rate (AL), critic network learning rate (CL), mini-batch size (BS), and exploration noise (EN). All remaining TD3 parameters—including network architecture, discount factor, target-network smoothing coefficient, replay buffer size, and policy-update frequency—were kept fixed at literature-recommended values to isolate the effects of the tuned parameters. This design ensures interpretability of tuning outcomes and prevents confounding interactions among secondary hyperparameters. At the end of each trial, the trained actor policy was evaluated on the fixed validation set, and the average makespan across validation instances was returned to Optuna as the objective value. The tuning process seeks:

$$\beta^* = \arg \min_{\beta \in \nu} J(\beta) \quad (11)$$

Where, $\beta = (AL, CL, BS, EN)$, $J(\beta)$ is the validation makespan obtained using TD3 trained with hyperparameters β , and ν is predefined ranges for the selected hyperparameters, as shown in [Table 3](#).

Where, $\beta = (AL, CL, BS, EN)$, $J(\beta)$ is the validation makespan obtained using TD3 trained with hyperparameters β , and ν is predefined ranges for the selected hyperparameters, as shown in [Table 3](#).

7.2.3. Bayesian optimization mechanism

Optuna guided the search using the Tree-structured Parzen Estimator (TPE) algorithm. In this approach, completed trials are partitioned into “good hyperparameter configurations” ($l(\beta)$) and “bad hyperparameter configurations” ($g(\beta)$) based on a percentile threshold of observed objective values as follows:

$$l(\beta) = p(\beta | J(\beta) \leq J^*) \quad (12)$$

$$g(\beta) = p(\beta | J(\beta) > J^*) \quad (13)$$

Where J^* is the 25th percentile of objective values from previous trials. Probabilistic models are then constructed for both groups, and new hyperparameter configurations are sampled to maximize the expected improvement as follows:

$$\beta_{t+1} = \arg \max_{\beta} \left(\frac{l(\beta)}{g(\beta)} \right) \quad (14)$$

Then, the evaluation procedure for 50 trials is started through sampling hyperparameters $\beta_i = (AL_i, CL_i, BS_i, EN_i)$ and training TD3 for 1400 episodes. The learned policy is evaluated using a fixed validation set of five instances:

$$J(\beta_i) = \frac{1}{5} \sum_{k=1}^5 \text{Makespan}(\pi_{\beta_i}, \text{Instance}_k) \quad (15)$$

Finally, the likelihood models $l(\beta)$ and $g(\beta)$ are updated.

Table 3
Predefined Ranges of the key hyperparameters.

Parameter	Predefined range
AL	$[10^{-5}, 10^{-3}]$
CL	$[10^{-5}, 10^{-3}]$
BS	$[32, 256]$
EN	$[0.01, 0.3]$

Consequently, poorly performing regions of the hyperparameter space are progressively discarded, while promising regions are explored more intensively. This adaptive strategy enables efficient optimization without exhaustive enumeration, which would be computationally prohibitive given the cost of TD3 training.

7.2.4. Termination and robustness

The optimization was terminated after completing the predefined 50 trials, balancing solution quality with computational feasibility. The observed variation in solution quality across neighboring configurations was limited, indicating that the selected hyperparameters yield stable convergence and robust performance. This tuning strategy ensures that the final TD3 configuration minimizes expected makespan under realistic deployment conditions, emphasizes generalization across instances, and relies on statistically meaningful performance trends rather than isolated outcomes.

The optimal hyperparameters are listed in Table 4 and briefly described below.

- Units of the actor/critic network fully connected layer: It represents the number of neurons in each hidden layer of the actor and critic networks. This hyperparameter influences the capacity of the model for learning complicated representations.
- Actor network learning rate: It controls the step size for updating the weights of the actor network. This hyperparameter influences how fast the policy responds to new information.
- Critic network learning rate: It determines the step size for the critic network update, affecting stability and value function estimation accuracy.
- Discount factor: It represents the importance of future rewards in decision-making, with a value close to 1 preferring long-term rewards and a lower value preferring short-term rewards.
- Mini-batch size: It determines the number of experiences sampled from the replay buffer for each training iteration. This hyperparameter affects the stability and efficiency of learning.
- Target smooth factor: It controls the soft update of target networks to reduce variance in value estimation and improve training stability.
- Optimizer: It represents the optimization algorithm employed to update network parameters for efficient gradient descent and convergence.
- Exploration noise: It adds randomness to actions during training to encourage exploration, preventing the agent from selecting sub-optimal policies.
- Policy update frequency: It specifies how often the actor network is updated compared to the critic network. This hyperparameter balances policy learning with value function estimation.

Table 4
Parameter setting of TD3 algorithm.

Parameter	Value
Units of actor network fully connected layer	256, 256, 128, action dimension
Units of critic network fully connected layer	256, 256, 128, 1
Actor network learning rate	3×10^{-4}
Critic network learning rate	3×10^{-4}
Discount factor	0.99
Mini batch size	100
Target smooth factor	5×10^{-3}
Optimizer	Adam
Exploration noise	0.1
Policy update frequency	2
Policy clip	0.5
Training round	10^6
Size of experience replay buffer	10^6

- Policy clip: It limits the range of policy updates to prevent drastic policy changes, enhancing training stability and preventing divergence.
- Training round: It represents the total number of iterations of the model throughout training. This hyperparameter impacts the overall learning process and convergence.
- Size of experience replay buffer: It denotes the capacity of stored past experiences used for training, ensuring a diverse set of experiences for stable learning.

7.3. Results

The presented framework was implemented in Python using TensorFlow and PyTorch for deep reinforcement learning, along with NumPy and Pandas for data processing, and matplotlib for data visualization. Experiments were conducted on a Windows 10 system with an Intel Core i9-13900 K processor, 64 GB of RAM, and an NVIDIA RTX 4090 GPU. Fig. 4 depicts the reward per episode during the training of the proposed TD3-based model for solving the RCMPSP. Specifically, the single reward per episode is shown in Fig. 4(a). As illustrated, the initial training phase reflects high fluctuations and negative rewards, exhibiting the agent's exploration and suboptimal scheduling decisions. Once training progressed, the reward stabilized, indicating enhanced scheduling performance. Moreover, the average reward per episode is presented in Fig. 4(b), which initially decreases due to the trial-and-error learning process but finally increases, representing the ability of the agent to refine its policy over time. The overall upward trend in both figures denotes that the model effectively learns an improved scheduling policy, converging toward an optimal solution.

Furthermore, the optimal scheduling plan is depicted in Fig. 5. To assess the efficiency of the proposed TD3-based approach, its performance is compared with the baseline DDPG, a widely used RL algorithm for continuous allocation. The project finish times of both methods are reported in Table 5, showing that TD3 consistently achieves shorter completion times across all projects.

Additionally, to compare the proposed framework with a representative traditional approach, the dataset was applied to the learning-based rollout metaheuristic introduced by Wauters et al. (2015), identified as the best-performing method for makespan minimization in a recent benchmark study (Bredael & Vanhoucke, 2023). As shown in Table 6, the metaheuristic, when executed for 8 h, produced a schedule with a makespan of 8,478. While this result is competitive relative to DDPG (9,900), it remains substantially inferior to TD3 (6,192), demonstrating the effectiveness of the proposed framework on large-scale multi-project scheduling.

In terms of training cost, DDPG required approximately 5–7 h to converge, whereas TD3 required 6–8 h. The slightly longer training time of TD3 is due to its use of twin critics, delayed policy updates, and target smoothing, which increase computation per update but stabilize learning and reduce overestimation bias. These features enable TD3 to achieve faster and more reliable convergence to high-quality policies. Once trained, both RL methods generate schedules in seconds per instance, whereas the metaheuristic must be rerun from scratch for every new portfolio, requiring several hours. This distinction underscores the scalability advantage of RL: while training represents a one-time offline cost, inference enables near real-time scheduling, making TD3 particularly suitable for dynamic or repeated decision environments.

Furthermore, to statistically evaluate the observed performance differences, a Wilcoxon signed-rank test was conducted on the project completion times obtained by TD3 and the learning-based rollout metaheuristic. The results, summarized in Table 7, confirm that TD3 significantly outperforms the metaheuristic across all projects.

Moreover, the completion rate of activities for the two scheduling approaches is compared in Fig. 6. Particularly, Fig. 6(a) depicts the completion rate using the proposed TD3-based method, exhibiting a

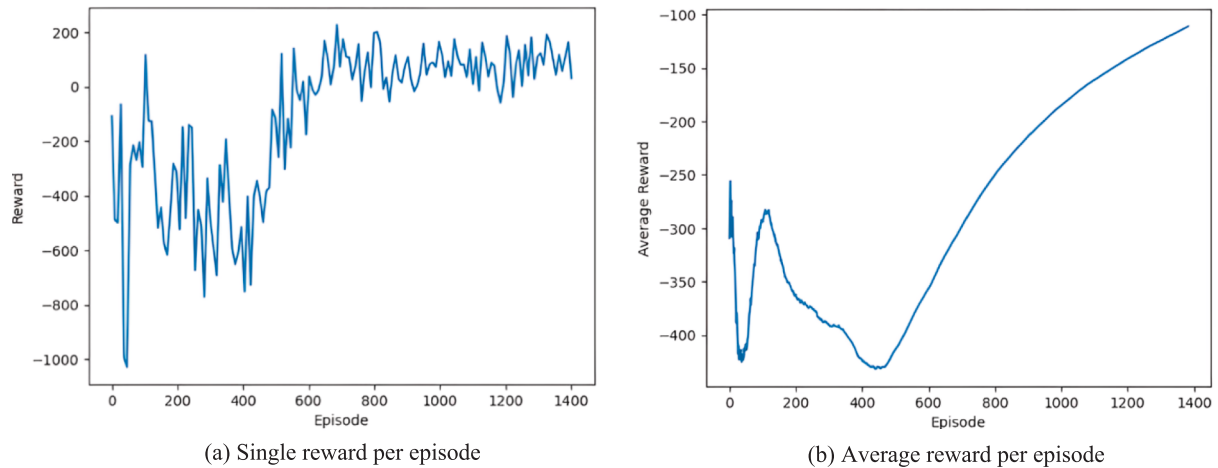


Fig. 4. TD3 algorithm reward across 1400 episodes.

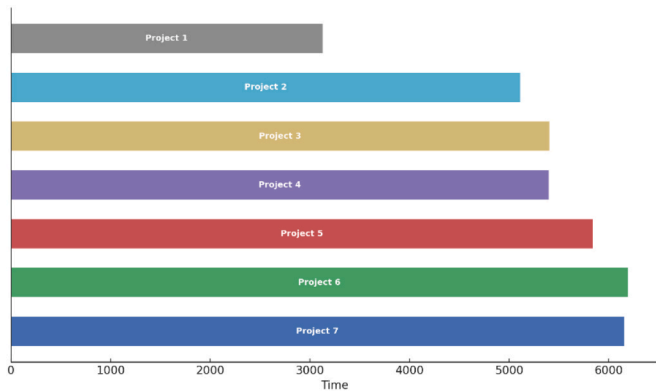


Fig. 5. The optimal Gantt chart of the dataset.

Table 5

Comparison of project finish times between DDPG and the proposed TD3 framework on the dataset.

Project No.	Finish Time (DDPG)	Finish Time (TD3)
1	4901	3128
2	8148	5110
3	8043	5403
4	8307	5395
5	9296	5839
6	9069	6192
7	9900	6154

Table 6

Performance comparison of the learning-based rollout metaheuristic, DDPG, and TD3 on the dataset.

Solution Method	Makespan	Training Time	Inference runtime per schedule
Learning-based Rollout Metaheuristic	8478	—	5–8 h
DDPG	9900	5–7 h	20–35 s
TD3	6192	6–8 h	15–25 s

steady progression, reaching a high percentage of completed sub-tasks in fewer steps. Fig. 6(b) illustrates the completion rate under the DDPG approach, which, while following a similar trend, needs more steps to achieve the same level of task completion. The comparison represents the superior efficiency of the proposed approach in

Table 7

Wilcoxon signed-rank test results comparing TD3 and the rollout metaheuristic on project completion times.

Comparison	Test Statistic (W)	P-Value	Effect Size (r)	Decision ($\alpha = 0.05$)
TD3 vs Learning-based Rollout Metaheuristic	0	0.0156	0.8957	Significant

accelerating task completion and optimizing resource utilization. Similarly, project completion rates for TD3 and DDPG are compared in Fig. 7, where TD3 (Fig. 7(a)) achieves faster and more structured completion with fewer steps, while DDPG (Fig. 7(b)) needs more steps to reach similar levels. This comparison suggests TD3's efficiency in improving scheduling performance.

Besides, resource utilization trends for four different resources under TD3 and DDPG are demonstrated in Fig. 8. It is shown that TD3 (Fig. 8 (a), 8(c), 8(e), and 8(g)) has a more structured and efficient usage pattern, achieving peak utilization faster and depleting resources smoothly. In contrast, DDPG (Fig. 8(b), 8(d), 8(f), and 8(h)) represents more fluctuations, indicating less stable resource allocation. Furthermore, Resource 2 and Resource 3 represent higher utilization levels compared to the other resources. This highlights their crucial role in task execution. Additionally, this suggests the advantage of TD3 in optimizing resource usage and reducing the overall makespan.

7.4. Sensitivity analysis and discussion

To further assess the effectiveness and robustness of the proposed TD3-based approach, a sensitivity analysis is conducted using three additional instances of varying sizes and complexities. The instances are selected from the well-known MPSPLIB dataset to ensure diversity in project structures and resource constraints. The detailed information of the selected instances is presented in Table 8. The analysis mainly focuses on the impact of problem size on scheduling performance while also considering the overload factor (OLF), which represents resource constraints and utilization efficiency.

The model has been applied to these three instances and compared against DDPG to evaluate its applicability to different sizes of problems and resource constraints. The completion rates of the projects for the three instances under TD3 (green) and DDPG (red) are shown in Fig. 9. In Fig. 9(a), corresponding to Instance 1 with a high OLF, TD3 achieves completion way before DDPG, thereby proving its capability in working under high overload. Fig. 9(b) represents Instance 2 with a low OLF; here, TD3 still shows a lead compared to DDPG. However, the gap

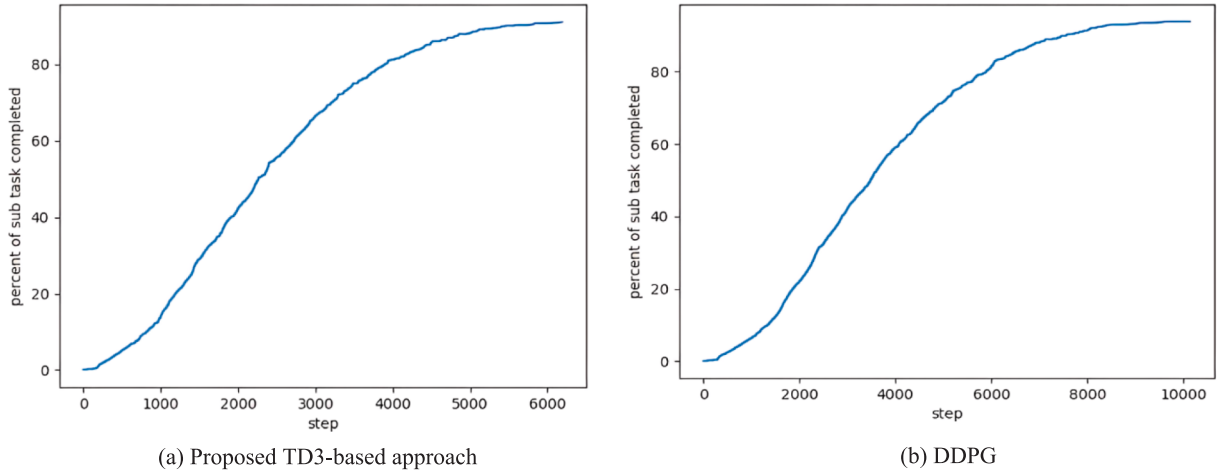


Fig. 6. Completion rate of activities in different solution methods.

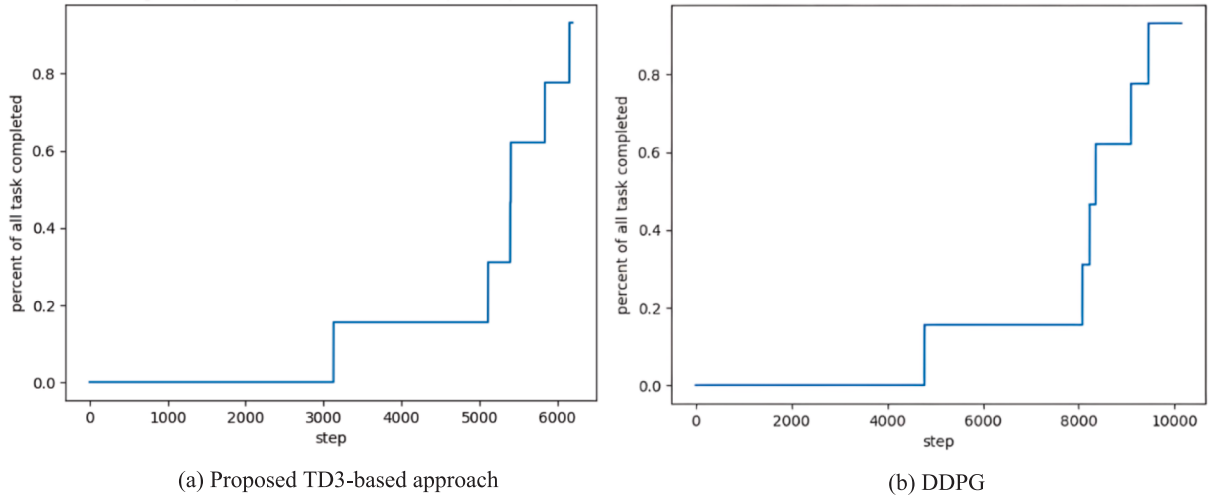


Fig. 7. Completion rate of projects in different solution methods.

shrinks, which means that the overload effect is less pronounced when the resources are less constrained. A high OLF for Instance 3 in Fig. 9(c) shows a step-like completion curve for both approaches, with TD3 still taking the clear advantage of faster completion. Overall, in all scenarios, TD3 outperforms the DDPG model almost always, especially in high OLF cases, thus confirming the strength of the model under resource-constrained situations. Further, the activity completion rates for the three instances under TD3 (green) and DDPG (red) are shown in Fig. 10. TD3 handles activities much faster and depicts much smoother completion than DDPG in all cases. In Fig. 10(a), TD3 maintains a constant lead, showing its best efficiency in terms of resolving the competing activities under resource constraints. There is a larger observable gap in Fig. 10(b), demonstrating TD3's advantage even when resource constraints are reasonably relaxed. Finally, TD3 outperformed DDPG in completing both structured and fast activities in Fig. 10(c). These results reconfirm the superiority of TD3 in both constrained and unconstrained environments.

Furthermore, Fig. 11 illustrates resource utilization trends for different instances across four resources (R1, R2, R3, R4) under TD3 (green) and DDPG (red). In Instance 1, subfigures (a), (b), (c), and (d) show that TD3 maintains a more structured and sustained resource utilization pattern, while DDPG demonstrates fluctuating peaks and a sudden decline in resource usage. For Instance 2, depicted in subfigures (e), (f), (g), and (h), TD3 continues to manage resources more

effectively, specifically for R1 and R3, whereas DDPG shows erratic spikes and inefficient allocation. In Instance 3, as shown in (i), (j), (k), and (l), TD3 again achieves a more controlled utilization trend, ensuring a smoother depletion of resources, while DDPG shows instability and steep drops in usage. In all instances, TD3 provides better resource management by maintaining a balanced allocation, preventing abrupt exhaustion, and enhancing scheduling efficiency, especially under varying problem sizes and overload factors.

Furthermore, to complement the sensitivity analysis, the computational complexity is evaluated. The state representation grows linearly with the total number of activities and the number of resources, while the feasible action set at each decision step is bounded by the total number of activities. The dominant cost per step is a forward pass through the actor and critic networks, which remains constant with respect to problem size. Over one training episode consisting of approximately as many steps as activities, the complexity therefore grows proportionally with the number of activities. Across multiple training episodes, the overall cost increases linearly with both the number of episodes and activities, while the experience replay buffer introduces an additional memory requirement proportional to its capacity. In contrast, the inference phase requires only a single forward pass per decision, i.e., a constant cost per step, enabling near real-time schedule construction even for large multi-project instances. These results clarify the scalability of the proposed framework and support the

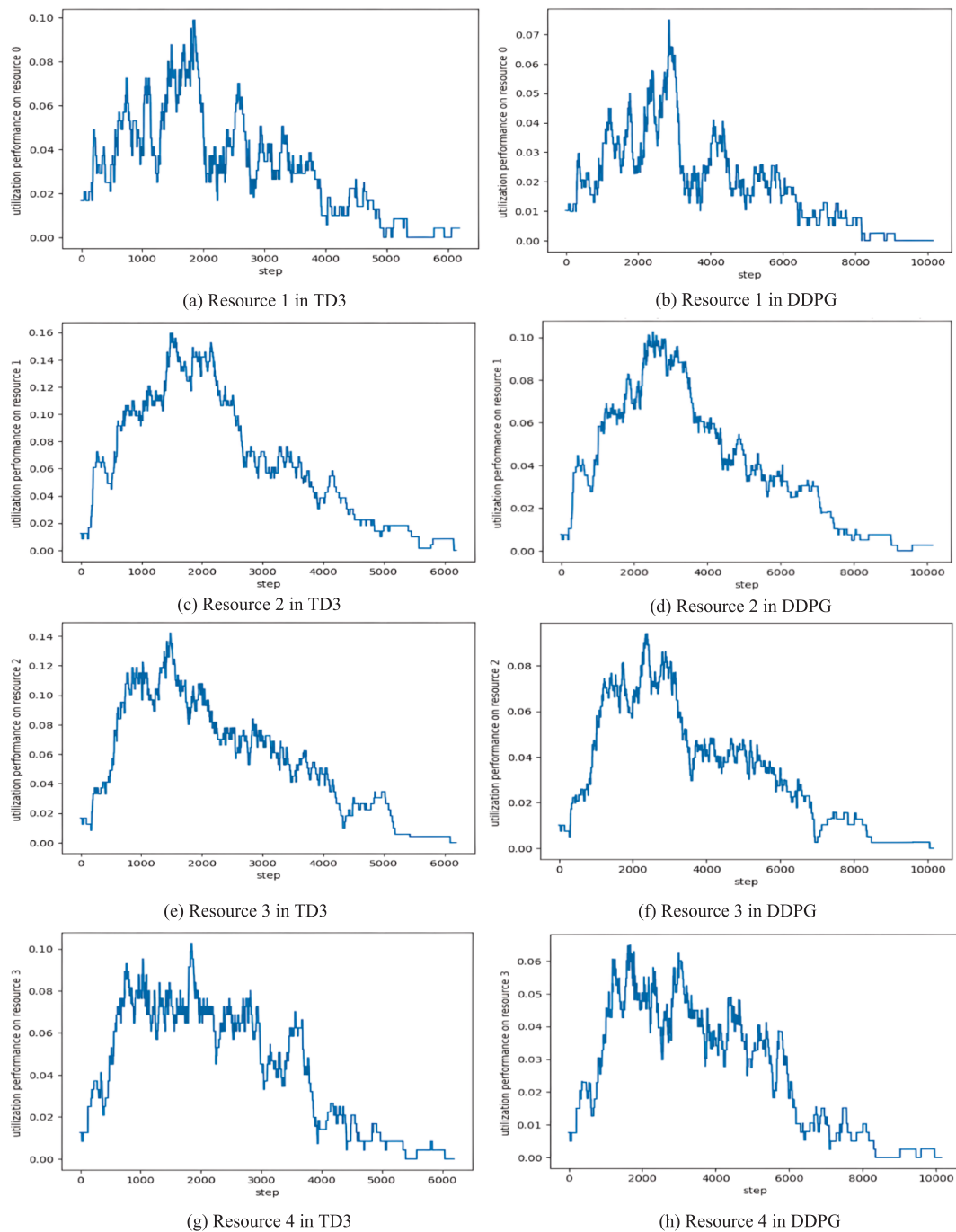


Fig. 8. Resource utilization rate.

stability observed in the sensitivity analysis.

7.5. Managerial and practical implications

According to the proposed solution method, results, and analysis, some practical and managerial insights are captured, presented as follows:

- TD3 relies heavily on a large amount of stored experiences to refine its policy and improve decision-making. A lack of adequate data for training can lead to a suboptimal scheduling solution, as the model may struggle to generalize across different project situations, such as software development projects with multiple interdependent tasks. Hence, project

managers should ensure comprehensive data collection and appropriate historical record-keeping of resource allocations, project schedules, and completion times. The quality, granularity, and representativeness of training data strongly affect

Table 8

Information of instances used for the sensitivity analysis.

No.	Instance Name	Number of Projects	Number of Activities	Number of Global Resources	OLF
1	mp_j30_a2_nr1	2	64	4	High
2	mp_j30_a5_nr1	5	160	4	Low
3	mp_j30_a5_nr3	5	160	4	High

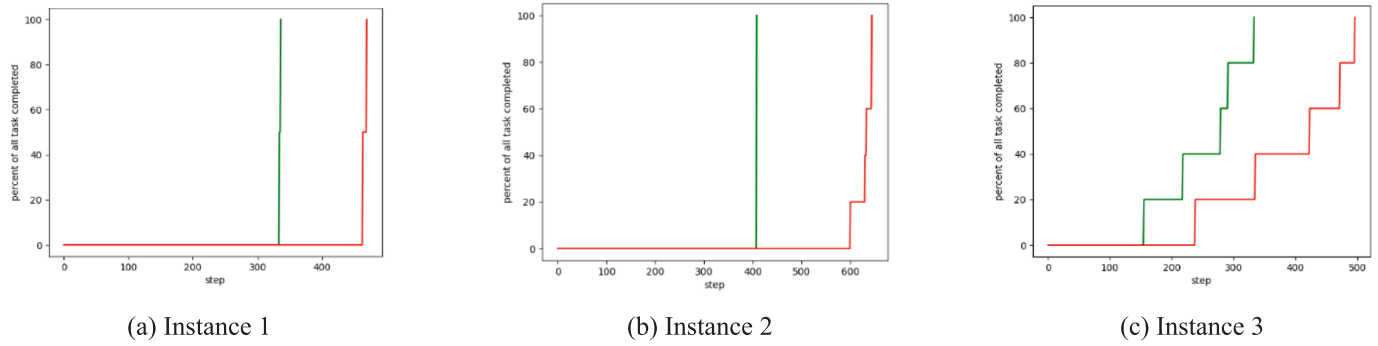


Fig. 9. Comparing projects completion rates of instances under TD3 with DDPG.

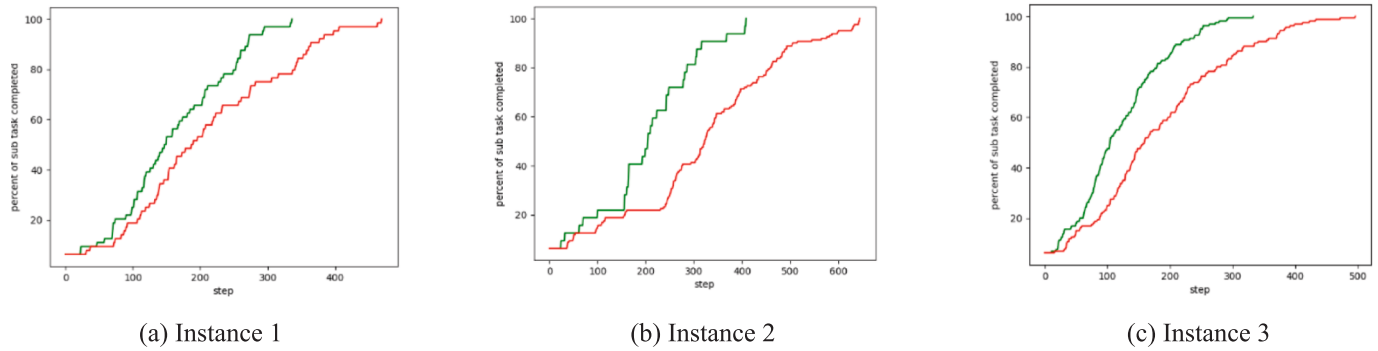


Fig. 10. Comparing the completion rates of activities of instances under TD3 with DDPG.

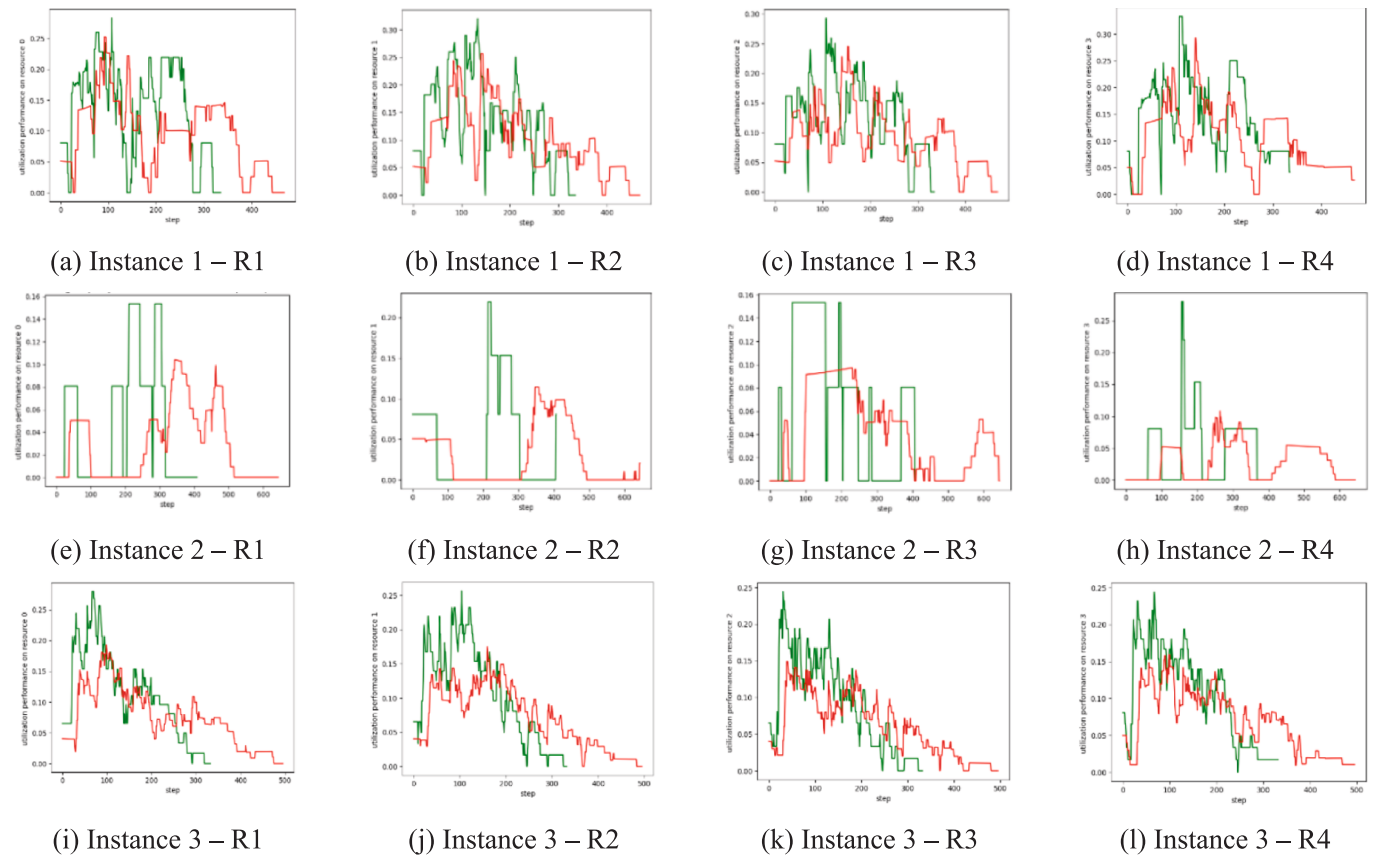


Fig. 11. Comparing resource graphs of instances under TD3 and DDPG.

performance; biased or incomplete records can misguide the learned policy. Moreover, the reward function must accurately encode project objectives (e.g., minimizing makespan while smoothing resource utilization); poorly specified rewards can bias exploration and degrade solution quality. Maintaining high-quality datasets and validated reward definitions therefore, materially improves TD3's effectiveness and generalizability.

- The effectiveness of TD3 in project scheduling relies not only on the algorithm itself but also on the computational resources available for processing large-scale scheduling environments. TD3 greatly benefits from GPU acceleration for fast neural network computations, particularly when handling complex, large-scale scheduling problems. However, for smaller instances, it can still function effectively on CPUs. For large cases, adequate GPU memory and throughput (for replay buffers and twin-critic updates) are required to avoid slow convergence and unstable learning. Conversely, right-sizing infrastructure to problem scale supports stable training and real-time inference. Decision-makers must evaluate the size of their scheduling problems and invest in appropriate computing infrastructure to ensure efficient execution and real-time adaptability. Allocating insufficient computational infrastructure may slow down the model's learning process and limit its effectiveness in optimizing schedules.
- The results demonstrate that resource utilization is not maximized in all cases, which highlights inefficiencies in scheduling. Project managers should carefully analyze the project scheduling plan, examine precedence constraints between activities, and leverage parallel activity execution to reduce overall project duration. Appropriate activity sequencing and workload balancing can prevent idle resources and unnecessary project delays. Moreover, integrating real-time monitoring and adaptive scheduling strategies will help dynamically adjust resource allocations based on project requirements, ensuring that activities are executed efficiently.
- Since the utilization of different resources varies significantly, an effective resource prioritization strategy is important for optimizing scheduling in multi-project environments. Portfolio managers must recognize critical resources that have the highest impact on project timelines and allocate them accordingly to ensure smooth project execution. This requires prioritizing essential resources for high-impact activities, balancing workload distribution, and mitigating bottlenecks that may arise from resource shortages. Implementing intelligent resource scheduling tools and predictive analytics can further improve decision-making, helping decision-makers allocate resources efficiently while maintaining project deadlines and cost-effectiveness.

8. Conclusions and future directions

This paper introduced a new deep reinforcement learning method for solving the RCMPSP, leveraging the TD3 algorithm to address the complexities of continuous allocation in scheduling decisions. By modeling RCMPSP as an MDP and employing TD3, the presented approach effectively captures the dynamic nature of multi-project environments, ensuring adaptive and optimized resource allocation. Unlike traditional heuristic and exact methods, TD3 autonomously learns optimal scheduling policies without requiring extensive manual tuning, making it well-suited for real-world project management challenges.

To evaluate the effectiveness of the proposed method, a well-established benchmark dataset, MPSPLIB, was employed, which allows for a fair comparison with other methods. Furthermore, a sensitivity analysis was performed on different problem sizes to evaluate the robustness of the proposed TD3-based method. The experimental results

reveal that TD3 outperforms alternative RL algorithms, including DDPG, in terms of decision-making efficiency, learning stability, and makespan reduction. In addition, TD3 achieved markedly better makespan performance than a state-of-the-art rollout metaheuristic method while also requiring significantly less runtime at inference. The findings highlight TD3's superior ability to optimize resource allocation, accelerate activity completion, and enhance overall scheduling performance, making it a promising solution for large-scale multi-project scheduling.

The findings of this study suggest several important managerial implications for project managers and practitioners. Since TD3 requires a substantial number of state-action transitions to learn effective scheduling policies, decision-makers should ensure access to detailed historical data on activity durations, precedence relations, and resource availability. In addition, the quality and consistency of collected data are critical, as noisy or incomplete records may reduce policy generalizability. Besides, computational resource allocation should align with problem size, utilizing GPUs for large-scale scheduling and CPUs for smaller environments. Moreover, optimizing activity sequencing and leveraging parallel activities can help minimize idle resources and reduce the overall makespan. Furthermore, strategic prioritization of resource allocation is pivotal to preventing bottlenecks and ensuring efficient project execution.

One of the primary limitations is that TD3 needs a large number of interactions with the environment to learn an effective policy, making training highly expensive in terms of both time and computational infrastructure. The method also shows sensitivity to the quality of the reward function, as poorly designed rewards can bias learning and degrade solution quality. Additionally, storing large replay buffers and training multiple neural networks requires high memory and processing power, making TD3 resource-intensive. Moreover, the present study focuses on the classical formulation of the RCMPSP, which may limit direct applicability to more complex real-world project environments. For future research, exploring hybrid RL models that integrate TD3 with other heuristic-based methods could further improve solution quality. Extending the framework to incorporate additional practical features such as multi-mode or preemptive activities, non-renewable resources, and stochastic parameters represents a promising direction for future studies. Adding local resources to the RCMPSP and solving it by DRL methods is another future research suggestion. Finally, employing other RL-based techniques for fine-tuning hyperparameters could improve convergence speed and scheduling performance.

CRediT authorship contribution statement

Sajad Soltan: Writing – original draft, Validation, Software, Methodology, Formal analysis, Data curation, Conceptualization. **Maryam Ashrafi:** Writing – review & editing, Validation, Supervision, Methodology, Conceptualization. **Ehsan Nazerfard:** Writing – review & editing, Visualization, Validation, Software, Methodology, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

- Ahmeti, A., & Musliu, N. (2025). Hybridizing constraint programming and meta-heuristics for multi-mode resource-constrained multiple projects scheduling Problem. *Journal of Heuristics*, 31(1), 2.

- Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. In *In Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining* (pp. 2623–2631).
- Araujo, J. A., Santos, H. G., Gendron, B., Jena, S. D., Brito, S. S., & Souza, D. S. (2020). Strong bounds for resource constrained project scheduling: Preprocessing and cutting planes. *Computers & Operations Research*, 113, Article 104782.
- Asadujjaman, M., Rahman, H. F., Chakraborty, R. K., & Ryan, M. J. (2021). Resource constrained project scheduling and material ordering problem with discounted cash flows. *Computers & Industrial Engineering*, 158, Article 107427.
- Asadujjaman, M., Rahman, H. F., Chakraborty, R. K., & Ryan, M. J. (2024). Supply chain integrated resource-constrained multi-project scheduling problem. *Computers & Industrial Engineering*, 194, Article 110380.
- Barnawi, A., Kumar, N., Budhiraja, I., Kumar, K., Almansour, A., & Alzahrani, B. (2022). Deep reinforcement learning based trajectory optimization for magnetometer-mounted UAV to landmine detection. *Computer Communications*, 195, 441–450.
- Bredael, D., & Vanhoucke, M. (2023). Multi-project scheduling: A benchmark analysis of metaheuristic algorithms on various optimisation criteria and due dates. *European Journal of Operational Research*, 308(1), 54–75.
- Bredael, D., & Vanhoucke, M. (2024). A genetic algorithm with resource buffers for the resource-constrained multi-project scheduling problem. *European Journal of Operational Research*, 315(1), 19–34.
- Cai, H., Bian, Y., & Liu, L. (2024). Deep reinforcement learning for solving resource constrained project scheduling problems with resource disruptions. *Robotics and Computer-Integrated Manufacturing*, 85, Article 102628.
- Cai, J., Peng, Z., Ding, S., Wang, Z., & Wei, Y. (2023). A problem-specific parallel pareto local search for the reactive decision support of a special RCPSP extension. *Complex & Intelligent Systems*, 9(6), 7055–7073.
- Cao, J., Harrold, D., Fan, Z., Morstyn, T., Healey, D., & Li, K. (2020). Deep reinforcement learning-based energy storage arbitrage with accurate lithium-ion battery degradation model. *IEEE Transactions on Smart Grid*, 11(5), 4513–4521.
- Chao, Y., Zhuang, C., Guo, H., & Liu, J. (2026). A genetic programming hyper-heuristic with whale optimization algorithm for the dynamic resource-constrained multi-project scheduling problems. *Expert Systems with Applications*, 295, Article 128881.
- Chau, T., Nguyen, M. T., Ngo, D. V., Nguyen, A. D. T., & Do, T. H. (2022, December). Deep reinforcement learning methods for automation forex trading. In 2022 RIVF International Conference on Computing and Communication Technologies (RIVF) (pp. 671–676). IEEE.
- Chen, S., Fang, S., & Tang, R. (2019). A reinforcement learning based approach for multi-projects scheduling in cloud manufacturing. *International Journal of Production Research*, 57(10), 3080–3098.
- Chen, S., Tang, B., & Wang, K. (2023). Twin delayed deep deterministic policy gradient-based intelligent computation offloading for IoT. *Digital Communications and Networks*, 9(4), 836–845.
- Chen, T., & Ju, C. (2015). Comparative analysis of swarm intelligence and heuristic priority rules for solving multi-project scheduling problem. *International Journal of Computing Science and Mathematics*, 6(6), 581–599.
- Chen, Y., Han, S., Chen, G., Yin, J., Wang, K. N., & Cao, J. (2023). A deep reinforcement learning-based wireless body area network offloading optimization strategy for healthcare services. *Health Information Science and Systems*, 11(1), 8.
- Choi, J., Reaff, M. J., & Lee, J. H. (2007). AQ-Learning-based method applied to stochastic resource constrained project scheduling with new project arrivals. *International Journal of Robust and Nonlinear Control: IFAC-Affiliated Journal*, 17(13), 1214–1231.
- Dankwa, S., & Zheng, W. (2019). Twin-delayed ddpg: A deep reinforcement learning technique to model a continuous movement of an intelligent robot agent. In *In Proceedings of the 3rd international conference on vision, image and signal processing* (pp. 1–5).
- Ding, H., Zhuang, C., & Liu, J. (2023). Extensions of the resource-constrained project scheduling problem. *Automation in Construction*, 153, Article 104958.
- Domínguez-Barbero, D., García-González, J., & Sanz-Bobi, M.Á. (2023). Twin-delayed deep deterministic policy gradient algorithm for the energy management of microgrids. *Engineering Applications of Artificial Intelligence*, 125, Article 106693.
- Dong, H., Dong, H., Ding, Z., Zhang, S., & Chang, T. (2020). *Deep Reinforcement Learning* (Vol. 24). Singapore: Springer Singapore.
- Farahmand-Mehr, M., & Mousavi, S. M. (2025). Resource-constrained multi-project scheduling problems considering time-dependent reliability of resources: A new immune genetic local search algorithm. *Kybernetes*.
- Fradi, A., & Benali, M. (2024, September). Multi-project Scheduling Problem with Limited Preemption and Ordinary Feeding Precedence Relations. In *IFIP International Conference on Advances in Production Management Systems* (pp. 537–551). Cham: Springer Nature Switzerland.
- Fu, F., & Zhou, H. (2021). A combined multi-agent system for distributed multi-project scheduling problems. *Applied Soft Computing*, 107, Article 107402.
- Fujimoto, S., Hoof, H., & Meger, D. (2018, July). Addressing function approximation error in actor-critic methods. In *International conference on machine learning* (pp. 1587–1596). PMLR.
- Gao, G., Wen, Y., & Tao, D. (2022). Distributed energy trading and scheduling among microgrids via multiagent reinforcement learning. *IEEE transactions on neural networks and learning systems*, 34(12), 10638–10652.
- Geiger, M. J. (2017). A multi-threaded local search algorithm and computer implementation for the multi-mode, resource-constrained multi-project scheduling problem. *European Journal of Operational Research*, 256(3), 729–741.
- Goudarzi, E., Esmaeili, H., Parsa, K., & Asadzadeh, S. (2024). Energy-efficient resource-constrained multi-project scheduling problem with generalized precedence relations and multi-skilled resources. *The Journal of Supercomputing*, 80(10), 13837–13872.
- Hammler, P., Riesterer, N., & Braun, T. (2023). Fully dynamic reorder policies with deep reinforcement learning for multi-echelon inventory management. *Informatik Spektrum*, 46(5), 240–251.
- Hartmann, S., & Briskorn, D. (2022). An updated survey of variants and extensions of the resource-constrained project scheduling problem. *European Journal of operational research*, 297(1), 1–14.
- Hauder, V. A., Beham, A., Raggl, S., Parragh, S. N., & Affenzeller, M. (2020). Resource-constrained multi-project scheduling with activity and time flexibility. *Computers & Industrial Engineering*, 150, Article 106857.
- Homberger, J. (2007). A multi-agent system for the decentralized resource-constrained multi-project scheduling problem. *International Transactions in Operational Research*, 14(6), 565–589.
- He, N., Zhang, D. Z., & Yuce, B. (2022). Integrated multi-project planning and scheduling-a multiagent approach. *European Journal of Operational Research*, 302(2), 688–699.
- He, Y., He, Z., & Wang, N. (2021). Tabu search and simulated annealing for resource-constrained multi-project scheduling to minimize maximal cash flow gap. *Journal of Industrial & Management Optimization*, 17(5), 2451–2474.
- He, Y., Jia, T., & Zheng, W. (2023). Tabu search for dedicated resource-constrained multiproject scheduling to minimise the maximal cash flow gap under uncertainty. *European Journal of Operational Research*, 310(1), 34–52.
- Hu, S., Zhang, Z., Wang, S., Kao, Y., & Ito, T. (2019). A project scheduling problem with spatial resource constraints and a corresponding guided local search algorithm. *Journal of the Operational Research Society*, 70(8), 1349–1361.
- Hu, X., Li, J., Li, F., Wang, J., & Wang, Y. (2025). Priority rule-based heuristics for distributed multi-project scheduling considering global resource failures. *Journal of the Operational Research Society*, 1–20.
- Huang, B., Xie, J., & Yan, J. (2024). Inspection robot navigation based on improved TD3 algorithm. *Sensors*, 24(8), 2525.
- Issa, S. B., Patterson, R. A., & Tu, Y. (2021). Solving resource-constrained multi-project environment under different activity assumptions. *International Journal of Production Economics*, 232, Article 107936.
- Javanmard, S., Afshar-Nadjafi, B., Niaki, T. A., & S.. (2022). A bi-objective model for scheduling of multiple projects under multi-skilled workforce for distributed load energy usage. *Operational Research*, 22(3), 2245–2280.
- Jędrzejowicz, P., & Ratajczak-Ropel, E. (2014). Reinforcement learning strategies for A-team solving the resource-constrained project scheduling problem. *Neurocomputing*, 146, 301–307.
- Jędrzejowicz, P., & Ratajczak-Ropel, E. (2015, May). Reinforcement learning strategy for solving the MRCPSP by a team of agents. In *International Conference on Intelligent Decision Technologies* (pp. 537–548). Cham: Springer International Publishing.
- Jiang, C., Li, X., Lin, J. R., Liu, M., & Ma, Z. (2023). Adaptive control of resource flow to optimize construction work and cash flow via online deep reinforcement learning. *Automation in Construction*, 150, Article 104817.
- Jiang, D., Cai, Z., Liu, Z., Peng, H., & Wu, Z. (2022). An integrated tracking control approach based on reinforcement learning for a continuum robot in space capture missions. *Journal of Aerospace Engineering*, 35(5), Article 04022065.
- Joo, B., & Chua, P. (2017). Multimode resource-constrained multi-project scheduling with ad hoc activity splitting. In 2017 IEEE International conference on industrial engineering and engineering management (IEEM) (pp. 2261–2265). IEEE.
- Kim, H. J., & Lee, J. H. (2023). Look-ahead based reinforcement learning for robotic flow shop scheduling. *Journal of Manufacturing Systems*, 68, 160–175.
- Kim, J. G., & Lee, B. (2020). Automatic P2P energy trading model based on reinforcement learning using long short-term delayed reward. *Energies*, 13(20), 5359.
- Kong, F., & Chen, K. (2025). Three-phase CPLEX with quota auction mechanism for dedicated resource-constrained multi-project scheduling problem with calendar constraints. *Engineering, Construction and Architectural Management*.
- Kong, M., & So, J. (2023). Empirical analysis of automated stock trading using deep reinforcement learning. *Applied Sciences*, 13(1), 633.
- Krüger, D., & Scholl, A. (2010). Managing and modelling general resource transfers in (multi-) project scheduling. *OR Spectrum*, 32(2), 369–394.
- Kuhnle, A., May, M. C., Schäfer, L., & Lanza, G. (2022). Explainable reinforcement learning in production control of job shop manufacturing system. *International Journal of Production Research*, 60(19), 5812–5834.
- Lee, J. H., & Kim, H. J. (2022). Reinforcement learning for robotic flow shop scheduling with processing time variations. *International Journal of Production Research*, 60(7), 2346–2368.
- Lei, K., Guo, P., Zhao, W., Wang, Y., Qian, L., Meng, X., & Tang, L. (2022). A multi-action deep reinforcement learning framework for flexible Job-shop scheduling problem. *Expert Systems with Applications*, 205, Article 117796.
- Li, F., & Xu, Z. (2018). A multi-agent system for distributed multi-project scheduling with two-stage decomposition. *PLoS One*, 13(10), Article e0205445.
- Li, H., & Womer, N. K. (2015). Solving stochastic resource-constrained project scheduling problems by closed-loop approximate dynamic programming. *European Journal of Operational Research*, 246(1), 20–33.
- Li, H., Zhang, X., Sun, J., & Dong, X. (2023). Dynamic resource levelling in projects under uncertainty. *International Journal of Production Research*, 61(1), 198–218.
- Li, X., He, Z., & Wang, N. (2024). A branch-and-bound algorithm for the proactive resource-constrained project scheduling problem with a robustness maximization objective. *Computers & Operations Research*, 166, Article 106623.
- Liu, H., Zhang, J., Zhang, X., & Chen, Z. (2024). A new resource-constrained project scheduling problem with ladder-type carbon trading prices and its algorithm based on deep reinforcement learning. *Expert Systems with Applications*, 255, Article 124794.

- Liu, J., & Lu, M. (2019). Robust dual-level optimization framework for resource-constrained multiproject scheduling for a prefabrication facility in construction. *Journal of Computing in Civil Engineering*, 33(2), Article 04018067.
- Liu, W., Zhang, J., Vanhoucke, M., & Guo, W. (2025). Resource allocation models and heuristics for the multi-project scheduling with global resource transfers and local resource constraints. *Computers & industrial engineering*, 200, Article 110843.
- Min, C. H., & Song, J. B. (2019, June). End-to-end robot manipulation using demonstration-guided goal strategie. In 2019 16th International Conference on Ubiquitous Robots (UR) (pp. 159-164). IEEE.
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., & Hassabis, D. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533.
- Nekoueiian, R., Servranckx, T., & Vanhoucke, M. (2025). A dynamic learning-based genetic algorithm for scheduling resource-constrained projects with alternative subgraphs. *Applied Soft Computing*, 113316.
- Pan, L., Cai, Q., & Huang, L. (2020). Softmax deep double deterministic policy gradients. *Advances in Neural Information Processing Systems*, 33, 11767–11777.
- Phuntsho, T., & Gonsalves, T. (2024). Maximizing net present value for resource constraint project scheduling problems with payments at event occurrences using approximate dynamic programming. *Algorithms*, 17(5), 180.
- Pellerin, R., Perrier, N., & Berthaut, F. (2020). A survey of hybrid metaheuristics for the resource-constrained project scheduling problem. *European Journal of Operational Research*, 280(2), 395–416.
- Rajasekhar, N., Radhakrishnan, T. K., & Samsudeen, N. (2024). Decentralized multi-agent control of a three-tank hybrid system based on twin delayed deep deterministic policy gradient reinforcement learning algorithm. *International Journal of Dynamics and Control*, 12(4), 1098–1115.
- Ratajczak-Ropel, E. (2018). Experimental evaluation of agent-based approaches to solving multi-mode resource-constrained project scheduling problem. *Cybernetics and Systems*, 49(5–6), 296–316.
- Sadeghloo, M., Emami, S., & Divsalar, A. (2024). A Benders decomposition algorithm for the multi-mode resource-constrained multi-project scheduling problem with uncertainty. *Ann. Oper. Res.*, 339(3), 1637–1677.
- Sallam, K. M., Chakraborty, R. K., & Ryan, M. J. (2021). A reinforcement learning based multi-method approach for stochastic resource constrained project scheduling problems. *Expert Systems with Applications*, 169, Article 114479.
- Samende, C., Cao, J., & Fan, Z. (2022). Multi-agent deep deterministic policy gradient algorithm for peer-to-peer energy trading considering distribution network constraints. *Applied Energy*, 317, Article 119123.
- Sánchez, M. G., Lalla-Ruiz, E., Gil, A. F., Castro, C., & Voß, S. (2023). Resource-constrained multi-project scheduling problem: A survey. *European Journal of Operational Research*, 309(3), 958–976.
- Satic, U., Jacko, P., & Kirkbride, C. (2022). Performance evaluation of scheduling policies for the dynamic and stochastic resource-constrained multi-project scheduling problem. *International Journal of Production Research*, 60(4), 1411–1423.
- Satic, U., Jacko, P., & Kirkbride, C. (2024). A simulation-based approximate dynamic programming approach to dynamic and stochastic resource-constrained multi-project scheduling problem. *European Journal of Operational Research*, 315(2), 454–469.
- Schnell, A., & Hartl, R. F. (2016). On the efficient modeling and solution of the multi-mode resource-constrained project scheduling problem with generalized precedence relations. *OR Spectrum*, 38(2), 283–303.
- Sewak, M., & Sewak, M. (2019). Deterministic policy gradient and the ddpg: Deterministic-policy-gradient-based approaches. *Deep Reinforcement Learning: Frontiers of Artificial Intelligence*, 173–184.
- Shen, X. N., Minku, L. L., Marturi, N., Guo, Y. N., & Han, Y. (2018). A Q-learning-based memetic algorithm for multi-objective dynamic software project scheduling. *Information Sciences*, 428, 1–29.
- Song, W., Kang, D., Zhanf, J., & Xi, H. (2017). A multi-unit combinatorial auction based approach for decentralized multi-project scheduling. *Autonomous Agents and Multi-Agent Systems*, 31, 1548–1577.
- Specht, J. M., & Madlener, R. (2023). Deep reinforcement learning for the optimized operation of large amounts of distributed renewable energy assets. *Energy and AI*, 11, Article 100215.
- Speranza, M. G., & Vercellis, C. (1993). Hierarchical models for multi-project planning and scheduling. *European Journal of Operational Research*, 64(2), 312–325.
- Su, Z. X., Wei, H. Y., Zou, X., & Qi, J. X. (2019). Zero-one formulation for a partial resource-constrained project scheduling problem with generalized precedence relations. *Journal of Construction Engineering and Management*, 145(3), Article 04018142.
- Sumiea, E. H., Abdulkadir, S. J., Alhussian, H. S., Al-Selwi, S. M., Alqushaibi, A., Ragab, M. G., & Fati, S. M. (2024). *Deep deterministic policy gradient algorithm: A systematic review*. Heliyon.
- Sun, J., Zhang, F., Wang, R., & Si, J. (2021, September). Research on resource-constrained multi-project scheduling problem considering resource substitution based on ant colony optimization. In 2021 International Conference on E-Commerce and E-Management (ICECEM) (pp. 69-73). IEEE.
- Sung, I., Choi, B., & Nielsen, P. (2020). Reinforcement learning for resource constrained project scheduling problem with activity iterations and crashing. *IFAC-PapersOnLine*, 53(2), 10493–10497.
- Tan, Y., Shen, Y., Yu, X., & Lu, X. (2023). Low-carbon economic dispatch of the combined heat and power-virtual power plants: A improved deep reinforcement learning-based approach. *IET Renewable Power Generation*, 17(4), 982–1007.
- Toffolo, T. A., Santos, H. G., Carvalho, M. A., & Soares, J. A. (2016). An integer programming approach to the multimode resource-constrained multiproject scheduling problem. *Journal of Scheduling*, 19, 295–307.
- Torba, R., Dauzère-Pères, S., Yugma, C., Gallais, C., & Pouzet, J. (2024). Solving a real-life multi-skill resource-constrained multi-project scheduling problem. *Ann. Oper. Res.*, 338(1), 69–114.
- Türk, A., Yılmaz, Ö. F., & Özkök, M. (2025). Examining a multi-skilled project scheduling problem for shipyard industry through operational and tactical perspectives. *Computers & Operations Research*, 107150.
- Uysal, F., Sonmez, R., & Isleyen, S. K. (2021). A graphical processing unit-based parallel hybrid genetic algorithm for resource-constrained multi-project scheduling problem. *Concurrency and Computation: Practice and Experience*, 33(16), e6266.
- Van Den Eeckhout, M., Vanhoucke, M., & Maenhout, B. (2021). A column generation-based diving heuristic to solve the multi-project personnel staffing problem with calendar constraints and resource sharing. *Computers & Operations Research*, 128, Article 105163.
- Van der Beek, T., Souravias, D., van Essen, J. T., Pruyn, J., & Aardal, K. (2024). Hybrid differential evolution algorithm for the resource constrained project scheduling problem with a flexible project structure and consumption and production of resources. *European Journal of Operational Research*, 313(1), 92–111.
- Van Eynde, R., & Vanhoucke, M. (2020). Resource-constrained multi-project scheduling: Benchmark datasets and decoupled scheduling. *Journal of Scheduling*, 23(3), 301–325.
- Van Eynde, R., & Vanhoucke, M. (2022). New summary measures and datasets for the multi-project scheduling problem. *European Journal of Operational Research*, 299(3), 853–868.
- Villafañez, F., Poza, D., López-Paredes, A., Pajares, J., & Olmo, R. D. (2019). A generic heuristic for multi-project scheduling problems with global and local resource constraints (RCMPSP). *Soft Computing*, 23(10), 3465–3479.
- Vishal, M., Satija, Y., & Babu, B. S. (2021, December). Trading agent for the Indian stock market scenario using Actor-Critic based reinforcement learning. In 2021 IEEE international conference on computation system and information technology for sustainable solutions (CSITSS) (pp. 1-5). IEEE.
- Wang, S., Li, J., Jiao, Q., & Ma, F. (2024). Design patterns of deep reinforcement learning models for job shop scheduling problems. *Journal of Intelligent Manufacturing*, 1–19.
- Wang, Y., Zhou, X., Shi, Y., Zhou, C., Jiang, Q., & Zheng, Z. (2024). Twin delayed deep deterministic policy gradient based stochastic optimal dispatch of hydro and wind power considering the uncertainty of wind power in power-time dimensions. *International Journal of Electrical Power & Energy Systems*, 163, Article 110326.
- Wang, Z., Cai, B., Li, J., Yang, D., Zhao, Y., & Xie, H. (2023). Solving non-permutation flow-shop scheduling problem via a novel deep reinforcement learning approach. *Computers & Operations Research*, 151, Article 106095.
- Wauters, T., Verbeeck, K., De Causmaecker, P., & Vanden Berghe, G. (2015). A learning-based optimization approach to multi-project scheduling. *Journal of Scheduling*, 18, 61–74.
- Xie, F., Li, H., & Xu, Z. (2021). An approximate dynamic programming approach to project scheduling with uncertain resource availabilities. *Applied Mathematical Modelling*, 97, 226–243.
- Xing, Z., Chen, C., & Tiong, R. L. K. (2025). Learning-enhanced differential evolution for multi-mode resource-constrained multi-project scheduling problem in industrial prefabrication. *Buildings*, 15(12), 1996.
- Yang, H., Wang, Z., Gao, Y., & Zhou, W. (2024). Bi-objective multi-mode resource-constrained multi-project scheduling using combined NSGA II and Q-learning algorithm. *Applied Soft Computing*, 152, Article 111201.
- Yao, Z., Olson, J., & Yoon, H. S. (2021). Sensitivity analysis of reinforcement learning-based hybrid electric vehicle powertrain control. *SAE International Journal of Commercial Vehicles*, 14(02–14-03-0033), 409–419.
- You, W., Xu, Z., Yu, Y., & Zhao, S. (2024). A two-layer approach for solving robust decentralized multiproject scheduling problem with multi-skilled staff. *International Transactions in Operational Research*, 31(3), 1631–1670.
- Zhang, Z., & Chen, M. (2018). A bi-level multi-agent system model for decentralized multi-project scheduling of wind power plants. *Journal of Renewable and Sustainable Energy*, 10(3).
- Zhang, J., Li, L., Demeulemeester, E., & Zhang, H. (2024). A three-dimensional spatial resource-constrained project scheduling problem: Model and heuristic. *European Journal of Operational Research*, 319(3), 943–966.
- Zhang, X., & Zhu, G. Y. (2024). A literature review of reinforcement learning methods applied to job-shop scheduling problems. *Computers & Operations Research*, 106929.
- Zhang, Y., Zhang, C., Fan, R., Huang, S., Yang, Y., & Xu, Q. (2022). Twin delayed deep deterministic policy gradient-based deep reinforcement learning for energy management of fuel cell vehicle integrating durability information of powertrain. *Energy Conversion and Management*, 274, Article 116454.
- Zhao, Y., Hu, X., Wang, J., & Cui, N. (2024). A robust multi-project scheduling problem under a resource dedication-transfer policy. *Ann. Oper. Res.*, 337(1), 425–457.
- Zhao, S., & Xu, Z. (2021). New closed-loop approximate dynamic programming for solving stochastic decentralized multi-project scheduling problem with resource transfers. *Expert Systems with Applications*, 185, Article 115593.
- Zhao, X., Song, W., Li, Q., Shi, H., Kang, Z., & Zhang, C. (2022, December). A deep reinforcement learning approach for resource-constrained project scheduling. In

- 2022 IEEE Symposium Series on Computational Intelligence (SSCI) (pp. 1226-1234). IEEE.
- Zhao, Y., Luo, X., & Zhang, Y. (2024). The application of heterogeneous graph neural network and deep reinforcement learning in hybrid flow shop scheduling problem. *Computers & Industrial Engineering*, 187, Article 109802.
- Zheng, K., Jia, X., Chi, K., & Liu, X. (2022). DDPG-based joint time and energy management in ambient backscatter-assisted hybrid underlay CRNs. *IEEE Transactions on Communications*, 71(1), 441–456.
- Zhentao, H., Nanfang, C., Xuejun, H., & Mahaffey, M. E. (2023). Time-and resource-based robust scheduling algorithms for multi-skilled projects. *Automation in Construction*, 153, Article 104948.